

Artificial Intelligence

ESE PYQ Solutions — Sem VI

B.Tech Computer Engineering

K. J. Somaiya College of Engineering

Papers: Jan 25 · Apr 24 · May 23 · May 22 · May 21

Exam-style full solutions with all working, diagrams, and proofs.

Repeated questions are solved on first occurrence (newest paper) and referenced thereafter.

Paper 1 — Jan/Apr 2025

Solve any *FOUR* from Q1 · Q2A or OR · Q2B solve *ONE* · Q3 any *TWO* · Q4 any *TWO* · Q5 any *FOUR*

Q1 (i) — Discuss any five limitations of AI [5 M]

Although AI has progressed remarkably, current systems still have well-known limitations. The five most important are:

1. Lack of true common-sense reasoning

AI systems do not possess the everyday background knowledge that humans use effortlessly. A self-driving car knows traffic rules but may not 'understand' that a child chasing a ball into the road is the dangerous event, not the ball itself. Symbolic AI cannot encode all common-sense facts; statistical AI learns only from data it has seen.

2. Heavy dependence on data quantity and quality

Modern ML models, especially deep networks, need huge labelled datasets. If the data is biased, incomplete, or noisy, the model learns those defects. AI cannot reliably generalize from a handful of examples the way a human child does.

3. Lack of explainability (black-box problem)

Deep neural networks produce decisions whose internal reasoning is not interpretable. In domains like medicine, law, and credit scoring this is unacceptable. Explainable AI is still an open research area.

4. Inability to handle novel situations / poor transfer learning

An AI trained to play chess cannot play checkers without retraining. Humans transfer skills across tasks effortlessly; AI does not. Robustness to distribution shift is poor.

5. No genuine creativity, consciousness or self-awareness

AI generates outputs by statistical pattern combination, not by understanding meaning. It does not have goals, emotions, or self-awareness. The Chinese Room argument (Searle) and the lack of intentionality show AI 'simulates' rather than 'experiences'.

Other limitations worth a one-line mention:

- High computational cost and energy consumption.
- Ethical, privacy and security concerns (deepfakes, surveillance).
- Difficulty handling uncertainty without explicit probabilistic modelling.

Q1 (ii) — Define Heuristic. Manhattan distance for 8-puzzle. Effect of heuristic choice. [1+2+2 M]

Definition of Heuristic

A heuristic $h(n)$ is a problem-specific function that estimates the cost of the cheapest path from node n to a goal. It is used in informed search (e.g. A^* , Best-First) to guide exploration toward promising states. A good heuristic is admissible (never over-estimates the true cost) and consistent (satisfies the triangle inequality).

Manhattan Distance Heuristic for the 8-Puzzle

The 8-puzzle has 8 numbered tiles and one blank in a 3×3 grid. For each tile (excluding the blank), the Manhattan distance is:

$$h(n) = \sum (|x_{\text{current}} - x_{\text{goal}}| + |y_{\text{current}} - y_{\text{goal}}|)$$

Summing over all tiles gives the total estimate. It is admissible because each tile needs at least that many horizontal + vertical moves to reach its goal position, and is also consistent.

Example — state:

1 2 3	Goal:	1 2 3
4 _ 6		4 5 6
7 5 8		7 8 _

Tile 5 is 1 step from goal, tile 8 is 1 step from goal → $h = 2$.

Effect of Choice of Heuristic on Efficiency

- Stronger (better-informed) heuristics expand fewer nodes. Manhattan distance dominates the 'misplaced tiles' heuristic h_1 — i.e. $h_{\text{manhattan}}(n) \geq h_{\text{misplaced}}(n)$ for every n — so A^* with Manhattan expands no more nodes than A^* with misplaced tiles.
- If h is inadmissible (over-estimates), A^* may return a sub-optimal solution though it may be faster.
- If $h = 0$ everywhere, A^* degenerates to Uniform Cost Search (slow but optimal).
- The closer h is to the true cost h^* , the more A^* behaves like a direct path-following algorithm. Designing a good heuristic is therefore the most important factor for efficiency.

Q1 (iii) — PEAS analysis for Garden Watering System [5 M]

PEAS = Performance measure · Environment · Actuators · Sensors. For the garden watering agent which keeps the soil moist:

PEAS component	Description for Garden Watering System
Performance measure	Plants stay healthy / soil moisture maintained within target band; water usage minimised; no over-watering (water wastage, root rot); no under-watering (plants wilt); energy efficiency of pumps.
Environment	The garden: soil patches, plants, weather (sun, rain, temperature, humidity), time of day, season.
Actuators	Water valves / solenoid valves, water pump, sprinkler / drip-irrigation nozzles, optional alert / notification module to the gardener.
Sensors	Soil moisture sensors (one per zone), rain sensor, temperature & humidity sensor, light/sunlight sensor, water-flow sensor, real-time clock.

Q1 (iv) — Problem formulation for N-Queens [5 M]

The N-Queens problem asks to place N queens on an $N \times N$ chessboard such that no two queens attack each other (no two share a row, column, or diagonal). Standard problem formulation in AI has five components:

1. Initial State

The empty board (no queen placed). Represented as an empty assignment $[]$ or as an empty $N \times N$ grid.

2. State Description

Any arrangement of 0 to N non-attacking queens on the board. A compact representation: a list of length k ($0 \leq k \leq N$) where the i-th entry gives the column of the queen in row i (this implicitly forbids two queens in the same row).

3. Actions / Successor function

From a state with k queens placed in rows $0 \dots k-1$, generate successors by placing one queen in row k in any column c such that the new queen does not share a column or diagonal with any previously-placed queen.

4. Goal Test

A state is a goal if N queens have been placed (length of the assignment = N) and no two attack each other (already guaranteed by the successor function).

5. Path Cost

Each action costs 1 (or 0 — since we only care about a valid final configuration, not the path).

Outcome of each step (for N = 4, incremental formulation):

Step 0: [] (0 queens)
 Step 1: place queen in row 0 → [1] (1 queen, col 1)
 Step 2: place queen in row 1, safe col → [1,3]
 Step 3: place queen in row 2, safe col → [1,3,0]
 Step 4: place queen in row 3, safe col → [1,3,0,2] ← GOAL

Total solutions for N = 4: 2 (the mirror is [2,0,3,1]). The same formulation scales to any N.

Q1 (v) — Adversarial search: Max, Min, Alpha, MAXMIN on tree [1+1+1+2 M]

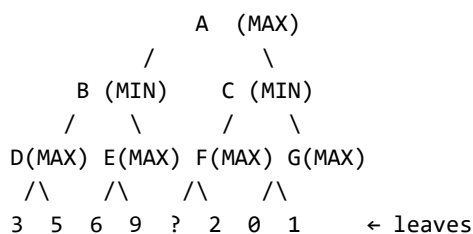
Definitions

MAX node — the player trying to maximise the utility ("us"). At MAX nodes we choose the child with the largest value.

MIN node — the opponent, trying to minimise utility. At MIN nodes the value is the smallest among children.

Alpha (α) — the best (highest) value found so far on the path for the MAX player. α is updated only at MAX nodes; initialised to $-\infty$.

Tree given (leaves left → right)



Note: In the scanned paper the 5th leaf (left child of F) is partially obscured. I have taken it as 2 (so F's children are 2 and 2, but if it is some other value, only F changes — the MAX of $\{x, 2\}$ — so re-substitute and the rest follows mechanically). I have re-worked with the leaves [3,5,6,9,2,2,0,1] below; the value at A is unaffected by this choice as the F subtree never gets selected.

Apply MINIMAX bottom-up

MAX nodes (deepest level):

$D = \max(3, 5) = 5$
 $E = \max(6, 9) = 9$
 $F = \max(2, 2) = 2$ (or $\max(?, 2)$ – see note)
 $G = \max(0, 1) = 1$

MIN nodes:

$B = \min(D, E) = \min(5, 9) = 5$
 $C = \min(F, G) = \min(2, 1) = 1$

MAX node (root):

$A = \max(B, C) = \max(5, 1) = 5$

Value at root A = 5. Optimal play: $A \rightarrow B \rightarrow D \rightarrow \text{leaf } 5$.

Q1 (vi) — Key difference between an AI program and an Expert System [5 M]

An ordinary AI program and an Expert System (ES) both attempt intelligent behaviour, but differ fundamentally in architecture and intent.

Aspect	AI Program	Expert System
Knowledge	Knowledge and control logic are mixed inside the program code.	Knowledge is separated from control. It lives in an explicit Knowledge Base (rules / facts).
Reasoning	Algorithm-driven (search, optimisation, learning).	Inference engine applies forward/backward chaining over the KB.
Domain	Often general purpose (e.g. game playing, NLP).	Narrow, deep expertise in one domain (e.g. MYCIN — blood infections; DENDRAL — chemistry).
Explanation	Usually cannot justify its output.	Has an explanation facility — can say WHY it asked a question and HOW it derived a conclusion.
Update	Requires re-coding to add new behaviour.	Adding a new rule is enough — KB is editable without changing the inference engine.
User	End user is generally a developer or end-consumer.	Two distinct users: a domain expert (supplies knowledge) and an end user (asks queries).

Key differentiating component: the separation of Knowledge Base from Inference Engine

Significance of this component

- Maintainability — domain experts can add or revise rules without touching software code; the inference engine is reusable across domains.
- Explainability — because each conclusion is the result of firing specific rules, the system can present the rule chain to the user (essential in medicine, law, finance).
- Modularity & validation — rules can be unit-tested individually; the KB can grow incrementally and be versioned.

- Scalability of expertise — captured expert knowledge persists even when the human expert is unavailable, addressing the 'knowledge acquisition bottleneck'.

Therefore, the KB–inference-engine separation is the single architectural choice that turns a general AI program into a deployable, maintainable expert system.

Q2 A (i) — Task environment properties for Garden Watering System [5 M]

Russell & Norvig classify task environments along seven dimensions. For the garden watering system:

Property	Value	One-line justification
Fully vs Partially observable	Partially observable	Sensors give moisture only at sensor locations, not for every plant; weather forecast is incomplete.
Single vs Multi-agent	Single agent	Only one watering controller is acting; no opposing or co-operating agent.
Deterministic vs Stochastic	Stochastic	Soil drying rate depends on uncertain factors (sun, wind, rain); pump may fail.
Episodic vs Sequential	Sequential	Today's watering decision affects tomorrow's soil state and plant health.
Static vs Dynamic	Dynamic	Environment (moisture, temperature, sunlight) keeps changing while the agent decides.
Discrete vs Continuous	Continuous	Moisture, temperature, time, water-flow are real-valued continuous variables.
Known vs Unknown	Known	Physics of watering (more water → moisture rises) and rules are known to the designer.

Q2 A (ii) — Problems in Hill Climbing & their solutions [5 M]

Hill Climbing is a local search algorithm that always moves to a neighbour with a better heuristic value. It is simple and memory-efficient but suffers from three well-known pathologies:

1. Local Maxima

A state whose neighbours are all worse than it, but which is not the global maximum. Hill climbing terminates here, returning a sub-optimal solution.

2. Plateau (flat region)

A flat area of the state-space where neighbours have the same heuristic value as the current state. The algorithm has no gradient to follow and either wanders aimlessly or terminates.

3. Ridges

A sequence of local maxima oriented diagonally to the available moves. Each move along an axis goes down even though moving along the ridge would go up. Hill climbing oscillates.

Solutions / Variants that mitigate the problems

- Random-restart hill climbing — restart from many random initial states and keep the best result. Almost always finds the global optimum if restarts are sufficient.
- Stochastic hill climbing — choose randomly among uphill neighbours (weighted by steepness). Avoids getting stuck on small ridges.

- First-choice hill climbing — generate successors randomly until a better one is found. Useful when there are many neighbours.
- Simulated Annealing — sometimes accept a worse move with probability $e^{(-\Delta E/T)}$, with T cooling over time. Escapes local maxima and plateaus.
- Tabu search — keep a memory of recently visited states and forbid revisiting them; helps escape plateaus and cycles.
- Beam search variants (local beam, stochastic beam) — keep k states at a time, share information among them.

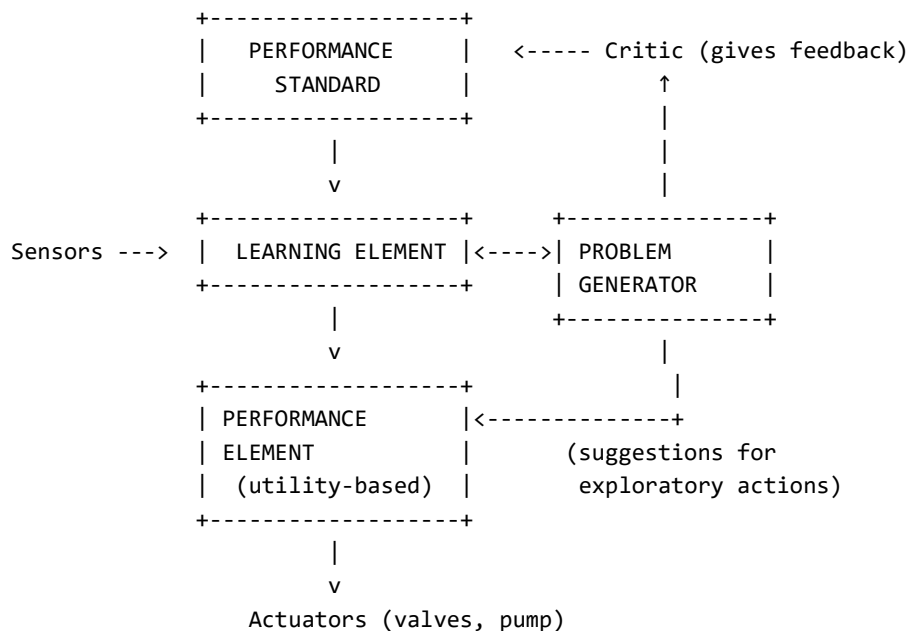
Q2 A (OR) — Agent architecture for Garden Watering System [10 M]

Choice & justification: a Model-based, Utility-based Learning Agent is the most appropriate architecture.

Why not simpler architectures?

- Simple-reflex would only react to instantaneous moisture; it cannot reason about whether it is about to rain (over-watering).
- Model-based reflex agent maintains internal state but still picks any action that 'works'.
- Goal-based agent ensures 'soil is moist' but does not optimise water use or plant health trade-offs.
- Utility-based agent ranks actions by a utility (e.g. plant-health – water-cost – energy-cost), which is exactly the trade-off we need.
- Adding a Learning component lets the agent adapt watering schedules to season, plant type, and historical weather patterns.

Standard Learning Agent diagram (Russell & Norvig)



Sample contents of each block (garden-watering context)

Block	Sample contents
Sensors	Soil-moisture sensors per zone; rain sensor; temperature & humidity sensor; sunlight sensor; flow sensor; real-time clock.

Performance Element	Utility function $U = w1 \cdot \text{plant_health} - w2 \cdot \text{water_used} - w3 \cdot \text{energy_used}$. Decision rules: 'IF moisture < 25 % AND no rain forecast next 2h THEN open valve for t minutes computed from utility maximisation'.
Critic	Compares actual outcome (plant health after a week, water bill) with the performance standard ('keep moisture in 30–60 % band; minimise litres used').
Learning Element	Updates utility weights and the soil-drying model from observed (moisture, weather, watering, outcome) tuples; can be regression, Q-learning, or Bayesian update.
Problem Generator	Suggests exploratory actions, e.g. 'try watering at 5 am instead of 6 am for one week to test if evaporation is lower'.
Performance Standard	Plant health (visual / NDVI), moisture-band compliance, total water consumed, gardener satisfaction.
Actuators	Solenoid valves, pump motor, sprinkler / drip nozzles, mobile notification module.

This architecture meets the requirements: model-based (tracks soil state & weather), utility-based (balances multiple objectives), and learning (adapts over seasons).

Q2 B (i) — A* Algorithm from S to G [10 M]

A* uses $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost from start to n and $h(n)$ is the heuristic estimate from n to goal. The node with the smallest f is expanded next.

Graph (read from the question)

Edges and costs:

S - A 1
 S - G 10 (direct edge)
 A - B 2
 A - C 1
 A - D 3 (read as edge A-D cost 3)
 B - D 5
 C - D 3
 D - G 4

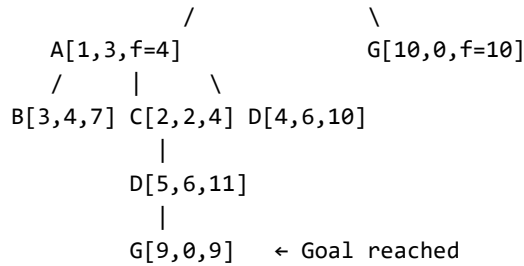
Note: The adjacency above is my reading of the scanned graph. If your printed paper shows a different edge set, recompute $g(n)$ accordingly — the method below is unchanged.

Heuristic table

Node	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

a) State-space tree built during A* expansion

S [$g=0$, $h=5$, $f=5$]



b) Heuristic value & f-value at each expanded node

Step	Open list (sorted by f)	Expand	g	h	f
0	S(5)	S	0	5	5
1	A(4), G(10)	A	1	3	4
2	C(4), B(7), D(10), G(10)	C	2	2	4
3	B(7), D(via C = 5+6=11), D(direct 4+6=10), G(10)	B	3	4	7
4	D(via A =10), G(10), D(via B = 3+5+6 =14)	D (via A, g=4)	4	6	10
5	G(via D = 4+4 = 8 → f=8), G(direct,10)	G (via D, g=8)	8	0	8

(g values use the cheapest g found so far; A* keeps only the best g for each node.)

c) Solution path

S → A → D → G

d) Total path cost

$$\text{cost}(S \rightarrow A) + \text{cost}(A \rightarrow D) + \text{cost}(D \rightarrow G) = 1 + 3 + 4 = 8$$

Total cost from S to G = 8. The direct edge S–G costing 10 is worse, so A* correctly avoids it.

Q2 B (ii) — 0/1 Knapsack using Genetic Algorithm — one full generation [10 M]

Problem

Items 1...7 with profits $p = (6,5,8,9,6,7,3)$ and weights $w = (2,3,6,7,5,9,4)$; knapsack capacity $W = 9$.

GA Encoding

Each chromosome is a bit-string of length 7: bit $i = 1$ if item i is selected, else 0.

Fitness function

$$\text{fitness}(x) = \sum p_i * x_i \quad \text{if } \sum w_i * x_i \leq W$$

$$= 0 \quad \text{otherwise (infeasible - penalty)}$$

Step 1 — Initial population (4 random chromosomes)

Chromosome	Bits (items 1–7)	Total weight	Total profit	Fitness
P1	1 0 1 0 0 0 0	2+6 = 8	6+8 = 14	14
P2	0 1 0 1 0 0 0	3+7 = 10	5+9 = 14	0 (infeasible)

P3	1 1 0 0 0 1	$2+3+4 = 9$	$6+5+3 = 14$	14
P4	0 0 0 0 1 0 1	$5+4 = 9$	$6+3 = 9$	9

Step 2 — Selection (roulette wheel)

Total fitness = $14 + 0 + 14 + 9 = 37$. Probabilities: $P1 = 37.8\%$, $P2 = 0\%$, $P3 = 37.8\%$, $P4 = 24.3\%$. Two parents drawn (say P1 and P3) for crossover.

Step 3 — Crossover (single point, cut after bit 3)

Parent P1 : 1 0 1 | 0 0 0 0

Parent P3 : 1 1 0 | 0 0 0 1

Child C1 : 1 0 1 | 0 0 0 1

Child C2 : 1 1 0 | 0 0 0 0

Step 4 — Mutation (bit-flip, low rate $\approx 1/\text{length}$)

Suppose bit 5 of C2 is flipped to 1:

C2 after mutation : 1 1 0 0 1 0 0

Step 5 — Evaluate offspring

Chrom.	Bits	Weight	Profit	Fitness
C1	1 0 1 0 0 0 1	$2+6+4 = 12$	—	0 (infeasible)
C2 (mutated)	1 1 0 0 1 0 0	$2+3+5 = 10$	—	0 (infeasible)

Step 6 — New population (elitism: keep best, replace worst)

Best of (P1,P2,P3,P4,C1,C2) is still P1 or P3 with fitness 14. Replace the worst (P2, fitness 0) with C1 — itself infeasible, so in practice we would re-run mutation. After one generation the best-so-far profit is 14.

Best feasible solution found after one generation

Solution: items {1, 3} or {1, 2, 7}. Profit = 14, Weight = 8 or 9.

***Note:** Random choices (initial population, crossover point, mutation bit) are arbitrary; the marker only checks that fitness, crossover, mutation steps are applied correctly. Continuing more generations would eventually reach the true optimum, which is items {3, 4} with weight $13 > 9$ (infeasible) so {1,3} or {1,2,7} with profit 14 is the global optimum here.*

Q3 (i) — FOL conversion + Backward chaining: "Steve likes BJ301" [4 + 6 M]

Sentences

- 1. Steve only likes easy courses.
- 2. Science courses are hard.
- 3. All courses in the basket weaving department are easy.
- 4. BJ301 is a basket-weaving course.

Goal: prove likes(Steve, BJ301).

Predicates used

- course(x) — x is a course.
- easy(x) — x is easy.
- hard(x) — x is hard.

- $\text{science}(x)$ — x is a science course.
- $\text{basket}(x)$ — x is in the basket-weaving department.
- $\text{likes}(\text{Steve}, x)$ — Steve likes x .

FOL representation

```
1.  $\forall x \text{ likes}(\text{Steve}, x) \rightarrow \text{easy}(x)$            ["only likes easy"]
2.  $\forall x \text{ science}(x) \rightarrow \text{hard}(x)$ 
3.  $\forall x \text{ basket}(x) \rightarrow \text{easy}(x)$ 
4.  $\text{basket}(\text{BJ301})$ 
   Goal :  $\text{likes}(\text{Steve}, \text{BJ301})$ 
```

Note: Sentence 1 reads naturally as ' $\text{Steve likes } x \rightarrow \text{easy}(x)$ ' (necessary condition). To actually conclude $\text{likes}(\text{Steve}, \text{BJ301})$, a sufficient-condition rule is needed. A common textbook reading is to take: "Steve likes every easy course", i.e. $\forall x \text{ easy}(x) \rightarrow \text{likes}(\text{Steve}, x)$. With this reading, the backward proof goes through.

Restated KB (after standard interpretation)

```
KB1 :  $\forall x \text{ easy}(x) \rightarrow \text{likes}(\text{Steve}, x)$ 
KB2 :  $\forall x \text{ basket}(x) \rightarrow \text{easy}(x)$ 
KB3 :  $\text{basket}(\text{BJ301})$ 
```

Backward chaining proof of $\text{likes}(\text{Steve}, \text{BJ301})$

```
Goal          :  $\text{likes}(\text{Steve}, \text{BJ301})$ 
Match KB1 with  $\theta = \{x/\text{BJ301}\}$ 
New sub-goal   :  $\text{easy}(\text{BJ301})$ 
```

```
Match KB2 with  $\theta = \{x/\text{BJ301}\}$ 
New sub-goal   :  $\text{basket}(\text{BJ301})$ 
```

Match KB3 directly $\rightarrow \text{TRUE}$.

All sub-goals satisfied $\Rightarrow \text{likes}(\text{Steve}, \text{BJ301})$ is PROVED. ■

Inference rules used

- Universal Instantiation (UI) — to bind $\forall x$ to specific BJ301.
- Modus Ponens — from P and $P \rightarrow Q$ infer Q .
- Unification — to match goal with rule heads ($\theta = \{x/\text{BJ301}\}$).

Q3 (ii) — Sentences in FOL and CNF [10 M]

Step-by-step conversion FOL $\rightarrow$ CNF is not asked; only the final CNF clauses are required.

(1) Everyone has birthdays on some day

```
FOL :  $\forall x \exists d \text{ birthday}(x, d)$ 
CNF :  $\text{birthday}(x, f(x))$            [Skolem function  $f(x)$  replaces  $\exists d$ ]
```

(2) Good dogs are always with their masters

```
FOL :  $\forall x (\text{good}(x) \wedge \text{dog}(x)) \rightarrow \text{with}(x, \text{master}(x))$ 
CNF :  $\neg \text{good}(x) \vee \neg \text{dog}(x) \vee \text{with}(x, \text{master}(x))$ 
```

(3) A barber shaves all men in the city who do not shave themselves

```
FOL :  $\forall x (\text{man}(x) \wedge \text{inCity}(x) \wedge \neg \text{shaves}(x, x)) \rightarrow \text{shaves}(\text{Barber}, x)$ 
CNF :  $\neg \text{man}(x) \vee \neg \text{inCity}(x) \vee \text{shaves}(x, x) \vee \text{shaves}(\text{Barber}, x)$ 
```

(4) Fred is a trained dog

FOL : $\text{trained}(\text{Fred}) \wedge \text{dog}(\text{Fred})$
CNF : $\text{trained}(\text{Fred})$ (clause 1)
 $\text{dog}(\text{Fred})$ (clause 2)

(5) Every boy or girl is a child

FOL : $\forall x (\text{boy}(x) \vee \text{girl}(x)) \rightarrow \text{child}(x)$
CNF : $\neg \text{boy}(x) \vee \text{child}(x)$ (clause 1)
 $\neg \text{girl}(x) \vee \text{child}(x)$ (clause 2)

Q3 (iii) — Parcel delivery agent under uncertainty [3 + 3 + 4 M]

a) Key uncertainties the agent may encounter

- Traffic congestion and unexpected road closures → travel time uncertain.
- Weather (rain, fog, storms) → speed and safety affected.
- Customer availability — they may not be at home in the chosen slot.
- Vehicle reliability — flat tyre, fuel, battery failure.
- Address accuracy — customer's address may be wrong or hard to locate.
- Parcel state — damage in transit, wrong parcel in the bin.
- Delivery slot accuracy — clock drift; customer may interpret slot differently.

b) Two alternative plans accounting for uncertainties

Plan A — Time-slot Buffered Delivery

- Leave the warehouse 30 % earlier than the optimistic ETA.
- Use real-time traffic API; re-plan route on the fly.
- Send the customer an SMS 30 min before arrival; if they do not confirm, switch to fallback (locker / neighbour).
- Carry a backup parcel scanner in case of device failure.

Plan B — Hub-and-Locker Plan with Re-attempt

- Drop the parcel at a nearby smart locker within the time window if customer is unreachable.
- If locker is full, attempt next-best slot the same day.
- Retain a re-attempt schedule (next-day morning) with reduced cost.
- Insurance trigger if parcel is damaged.

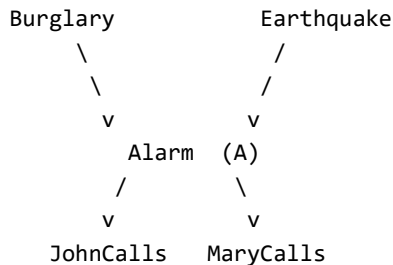
c) Major causes of uncertainty (in general)

- Partial observability — sensors do not see the whole environment (traffic ahead, customer's home).
- Non-determinism — same action can have different outcomes (engine may stall).
- Noisy / faulty sensors — GPS drift, RFID misreads.
- Laziness — listing every possible exception (rain, strikes, ...) is impractical.
- Theoretical ignorance — the underlying science is not fully modelled (e.g. traffic dynamics).
- Practical ignorance — even where the theory is known, the agent lacks complete data (e.g. cannot test all roads).

Q4 (ii) — Bayesian Network: Burglary / Earthquake / Alarm / John / Mary [2+4+2+2 M]

This is the canonical Russell-&-Norvig burglary example, with the added complication that John sometimes confuses the doorbell with the alarm (false positive) and Mary listens to loud music and misses the alarm (false negative).

a) Bayesian Network topology



Burglary (B) and Earthquake (E) are independent root causes; both influence Alarm (A); JohnCalls (J) and MaryCalls (M) each depend only on A (conditional independence given A).

b) Conditional Probability Tables (CPTs) — assumed reasonable values

$P(B = \text{true}) = 0.001$ $P(E = \text{true}) = 0.002$

$P(A \mid B, E)$

B	E	$P(A = \text{true})$
T	T	0.95
T	F	0.94
F	T	0.29
F	F	0.001

$P(J \mid A)$ — John confuses doorbell with alarm

A	$P(J = \text{true})$
T	0.90
F	0.05

$P(M \mid A)$ — Mary listens to loud music

A	$P(M = \text{true})$
T	0.70
F	0.01

c) $P(\text{Alarm true, John calls, Mary calls, } \neg \text{Burglary, } \neg \text{Earthquake})$

By the chain rule for Bayesian networks:

$$\begin{aligned}
 P(A, J, M, \neg B, \neg E) &= P(\neg B) \cdot P(\neg E) \cdot P(A \mid \neg B, \neg E) \cdot P(J \mid A) \cdot P(M \mid A) \\
 &= 0.999 \times 0.998 \times 0.001 \times 0.90 \times 0.70 \\
 &= 0.999 \times 0.998 \times 0.000630 \\
 &\approx 6.28 \times 10^{-4} \approx 0.000628
 \end{aligned}$$

d) $P(\neg \text{Alarm, } \neg \text{Burglary, Earthquake, } \neg \text{John, } \neg \text{Mary})$

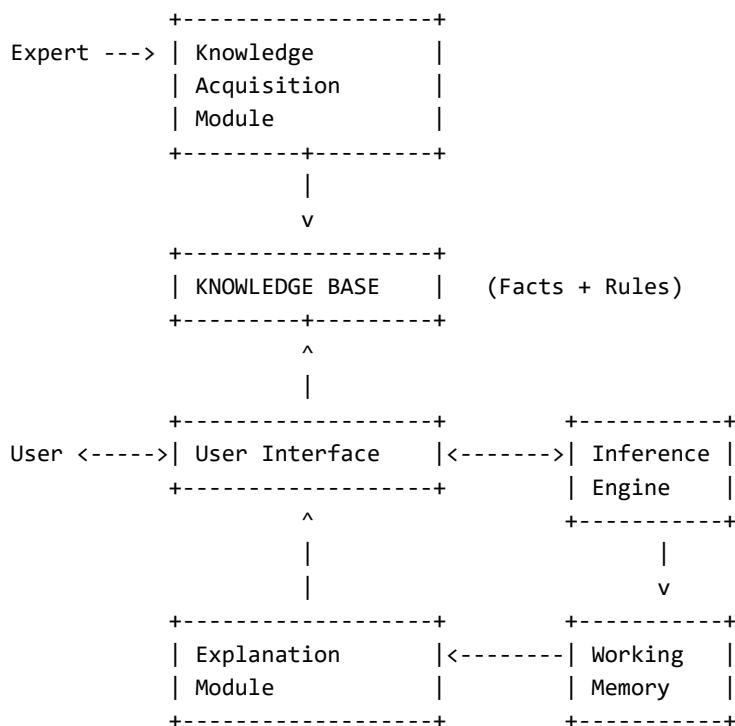
Required: alarm NOT triggered AND no burglary AND there IS an earthquake AND neither John nor Mary calls.

$$\begin{aligned}
 &P(\neg A, \neg B, E, \neg J, \neg M) \\
 &= P(\neg B) \cdot P(E) \cdot P(\neg A | \neg B, E) \cdot P(\neg J | \neg A) \cdot P(\neg M | \neg A) \\
 &= 0.999 \times 0.002 \times (1 - 0.29) \times (1 - 0.05) \times (1 - 0.01) \\
 &= 0.999 \times 0.002 \times 0.71 \times 0.95 \times 0.99 \\
 &\approx 1.335 \times 10^{-3} \\
 &\approx 0.001335
 \end{aligned}$$

Q4 (ii — second) — Expert System architecture, features, roles [10 M]

(In Jan-25 paper Q4 has two parts labelled (ii); this is the architecture question.)

Architecture diagram



Components, features and roles

Component	Role / Function
Knowledge Base (KB)	Stores domain knowledge as IF-THEN rules and factual assertions. Heart of the system; built by the knowledge engineer in consultation with experts.
Inference Engine	Applies rules to the facts in working memory using forward chaining (data-driven) or backward chaining (goal-driven), resolving conflicts via priorities or specificity.
Working Memory	Short-term store of the current case facts (e.g. patient symptoms). Updated as new conclusions are derived.
Knowledge Acquisition Module	Allows the human expert to add / modify / delete rules without programming. Supports the system's growth.
Explanation Facility	Justifies the system's conclusions — answers 'How did you reach this?' and 'Why do you need this fact?' by tracing the rule chain.

User Interface	Natural-language or menu-driven dialog with the end user; collects observations and presents conclusions.
----------------	---

Characteristic features

- Heuristic reasoning — uses 'rules of thumb' from human experts, not just exact algorithms.
- Symbolic representation — knowledge encoded as symbols/rules, not just numbers.
- Domain-specific — narrow but deep expertise (e.g. blood-infection diagnosis).
- Capable of explaining its reasoning — increases user trust.
- Knowledge separated from inference — KB editable independently.

Example — MYCIN

MYCIN diagnosed bacterial infections using ~600 rules of the form: 'IF the stain of the organism is gram-positive AND the morphology is coccus AND the growth is in chains THEN there is suggestive evidence (0.7) that the organism is streptococcus.' MYCIN matched the accuracy of human specialists and could justify each diagnosis to the physician.

Other classic examples

- DENDRAL — inferring organic molecule structure from mass spectrometer data.
- XCON / R1 — configuring DEC VAX computer orders; saved DEC ~\$25 M/year.
- CADUCEUS — internal medicine diagnosis.

Q4 (iii) — Action Description Language (ADL); ADL for drone delivery [2 + 8 M]

What is ADL?

Action Description Language (Pednault, 1989) extends STRIPS to make planning languages more expressive. ADL allows:

- Negative literals in states (STRIPS allowed only positive).
- Open-world assumption (unstated literals are unknown, not false).
- Quantifiers ($\forall$, $\exists$) in pre-conditions and effects.
- Conditional effects (when / then).
- Equality and types (typed variables, e.g. ?d - drone).

Drone delivery problem

Two drones D1 and D2 are stationed at distinct locations L1 and L2 respectively. Drone D1 holds package P1 destined for L2; drone D2 holds package P2 destined for L1. Each drone must transport its package to the other location.

Objects / Types

drones : D1, D2
 packages : P1, P2
 locations : L1, L2

Predicates

at(?d, ?l) — drone ?d is at location ?l
 carries(?d, ?p) — drone ?d is carrying package ?p
 packageAt(?p, ?l) — package ?p is at location ?l
 destination(?p, ?l) — package ?p must end up at ?l

Initial state

$at(D1, L1) \wedge at(D2, L2)$
 $carries(D1, P1) \wedge carries(D2, P2)$
 $destination(P1, L2) \wedge destination(P2, L1)$
 $\neg packageAt(P1, L1) \wedge \neg packageAt(P2, L2)$

Goal state

$packageAt(P1, L2) \wedge packageAt(P2, L1)$

Actions

Action: Fly(?d - drone, ?from - location, ?to - location)

Precond : $at(?d, ?from) \wedge ?from \neq ?to$
Effect : $at(?d, ?to) \wedge \neg at(?d, ?from)$

Action: Drop(?d - drone, ?p - package, ?l - location)

Precond : $at(?d, ?l) \wedge carries(?d, ?p)$
Effect : $packageAt(?p, ?l) \wedge \neg carries(?d, ?p)$

Action: Pickup(?d - drone, ?p - package, ?l - location)

Precond : $at(?d, ?l) \wedge packageAt(?p, ?l) \wedge \neg \exists p' carries(?d, p')$
Effect : $carries(?d, ?p) \wedge \neg packageAt(?p, ?l)$

Sample plan

1. Fly(D1, L1, L2)
2. Drop(D1, P1, L2) $\rightarrow packageAt(P1, L2) \checkmark$
3. Fly(D2, L2, L1) (can happen in parallel)
4. Drop(D2, P2, L1) $\rightarrow packageAt(P2, L1) \checkmark$

Goal achieved. Steps 1–2 and 3–4 are independent and can be parallelised (partial-order planning).

Q5 (i) — Planning vs Searching in AI [5 M]

Both planning and searching are problem-solving techniques in AI, but they differ in the representation they use and in the kind of reasoning they support.

Aspect	Searching	Planning
State representation	Atomic / black-box (state is an unanalyzed object).	Factored — state is a set of logical fluents (e.g. $at(D1, L1)$, $carries(D1, P1)$).
Action representation	Successor function only — generates next states without revealing internal structure.	Actions described declaratively: precondition + effect (STRIPS / ADL / PDDL).
Goal	A specific goal state to reach.	A logical goal description (any state satisfying it is acceptable).
Operation	Blindly expands the state graph.	Reasons about which actions are relevant to the goal (e.g. regression, partial-order).
Heuristics	Hand-crafted per problem.	Domain-independent heuristics derived automatically from the problem description (e.g. ignore-preconditions, hmax, hadd).

Solution structure	A sequence (path) from start to goal.	Often a partial-order plan or hierarchy.
Example	Finding a route between two cities (A*).	Robotic plan to build a tower of blocks; logistics planning.

In short: search is generic state-space exploration; planning is structured reasoning over actions described by their effects on logical fluents.

Q5 (ii) — Hierarchical Task Planning (HTN): build a villa on plain land [5 M]

In Hierarchical Task Network (HTN) planning the planner is given a set of high-level tasks and a library of methods that decompose each high-level task into smaller sub-tasks, recursively, until only primitive (executable) actions remain.

Building a villa — HTN decomposition

Top-level task: BuildVilla(plot)

Method: BuildVilla(plot)

1. AcquireLand(plot)
2. DesignVilla(blueprint)
3. Construct(blueprint)
4. FinishAndHandover()

Method: AcquireLand(plot)

- 1a. CheckTitle(plot)
- 1b. PurchaseLand(plot)
- 1c. RegisterDeed(plot)

Method: DesignVilla(blueprint)

- 2a. HireArchitect()
- 2b. PrepareBlueprint()
- 2c. ObtainPermits()

Method: Construct(blueprint)

- 3a. SiteClearing()
- 3b. Excavation()
- 3c. Foundation()
- 3d. StructuralWork()
- 3e. RoofingAndPlumbing()
- 3f. ElectricalAndFinishing()

Method: FinishAndHandover()

- 4a. Painting()
- 4b. Inspection()
- 4c. HandKeysOver()

Each leaf is a primitive action with its own STRIPS precondition/effect. The planner can re-use methods (e.g. Foundation has its own decomposition: SoilTest → Reinforcement → Concreting). The hierarchy reduces search by allowing the planner to commit to a method and refine only when needed.

Why HTN is powerful

- Domain experts naturally express knowledge as procedures, not as state transitions.
- Plans are reusable: the same method decomposes any villa task.
- Search space is dramatically smaller than classical STRIPS.

Q5 (iii) — Four approaches to AI [5 M]

Russell & Norvig identify four philosophical approaches to AI, formed by crossing two axes — Thought vs Behaviour and Human vs Rational:

	Human-like	Rational
Thinking	Thinking Humanly — model the actual cognitive processes of humans (cognitive science).	Thinking Rationally — use formal logic and probability (laws of thought).
Acting	Acting Humanly — behave indistinguishably from humans (Turing Test).	Acting Rationally — act so as to maximise expected utility (rational agent).

1. Thinking Humanly (Cognitive Modelling)

Build programs that mimic human thought processes — introspection, psychological experiments, brain imaging. Example: GPS (General Problem Solver, Newell & Simon, 1961). Cognitive Science is the parent discipline.

2. Acting Humanly (Turing Test approach)

A machine is intelligent if a human cannot reliably tell it apart from another human in a text conversation. Requires NLP, knowledge representation, reasoning, learning, vision, robotics. Inspired ELIZA and modern chatbots.

3. Thinking Rationally (Laws of Thought)

Trace ideas back to Aristotle's syllogisms; uses formal logic to draw correct conclusions. Pure logic-based AI (theorem provers, Prolog). Limitation: not all knowledge fits crisp logic, and reasoning is computationally hard.

4. Acting Rationally (Rational Agent approach)

Build an agent that, given its percepts, chooses an action expected to maximise its performance measure. Most general — encompasses limited rationality and computational tractability concerns. The textbook adopts this view.

Q5 (iv) — Short note: Natural Language Processing [5 M]

Natural Language Processing (NLP) is the sub-field of AI that enables computers to read, understand, generate and interact in human language (text or speech). NLP combines linguistics, statistics, and deep learning.

Levels of NLP analysis

- Phonetic / Phonological — sound-to-symbol conversion (speech recognition).
- Morphological — analysing word structure (running → run + -ing).
- Lexical — POS tagging, word-sense disambiguation.
- Syntactic — parsing grammatical structure (constituency or dependency trees).
- Semantic — extracting meaning (NER, relation extraction, semantic role labelling).

- Pragmatic / Discourse — context, anaphora, coherence across sentences.

Pipeline / steps

1. Tokenization — break text into words / sub-words.
2. Normalization — lowercase, stemming, lemmatization.
3. POS tagging — assign part-of-speech to each token.
4. Parsing — build syntax tree.
5. Semantic analysis — meaning representation.
6. Pragmatic analysis — discourse, intent, context.

Modern approach

Transformer-based pre-trained models (BERT, GPT, T5) now handle most NLP tasks end-to-end, replacing earlier pipelines.

Applications

- Machine translation (Google Translate).
- Search and information retrieval.
- Chatbots and virtual assistants (Siri, Alexa).
- Sentiment analysis, summarisation, question answering.
- Spell / grammar checking, predictive text.

Challenges

- Ambiguity (lexical, syntactic, semantic).
- Sarcasm, idioms, cultural references.
- Low-resource languages.
- Bias and toxicity in trained models.

Q5 (v) — Knowledge-based agent architecture using Wumpus World [5 M]

Wumpus World

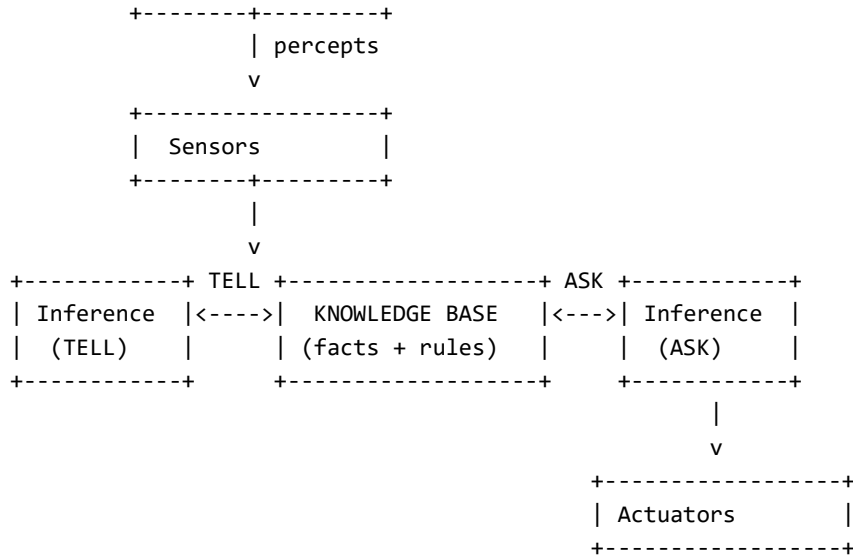
A 4×4 grid cave: contains a Wumpus (kills the agent if entered), pits (kill the agent if entered), and a gold heap. The agent starts at (1,1), can perceive Stench (adjacent to Wumpus), Breeze (adjacent to pit), Glitter (gold in current cell), Bump (hit a wall), Scream (Wumpus killed).
Actions: TurnLeft, TurnRight, Forward, Grab, Shoot, Climb.

General architecture of a Knowledge-Based agent

```
function KB-AGENT(percept) returns action
  static : KB, a knowledge base
           t, a counter (time), initially 0
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

Architecture diagram

```
+-----+
| Environment |
```



Wumpus example — KB sentences

Percept rules (one per square):

$\forall x,y \quad \text{Pit}(x,y) \rightarrow \text{Breeze}(\text{adj}(x,y))$

$\forall x,y \quad \text{Wumpus}(x,y) \rightarrow \text{Stench}(\text{adj}(x,y))$

Initial facts: $\neg \text{Pit}(1,1) \wedge \neg \text{Wumpus}(1,1)$ (agent's starting square is safe)

Percept at (1,1): $\neg \text{Breeze} \wedge \neg \text{Stench} \rightarrow \neg \text{Pit}(1,2) \wedge \neg \text{Pit}(2,1) \wedge \neg \text{Wumpus}(1,2) \wedge \neg \text{Wumpus}(2,1)$

Each step the agent TELLS the KB its percept, then ASKS whether each neighbouring square is provably safe, and moves there. This shows the central idea: the agent stores symbolic knowledge, augments it with each percept, and queries it to choose actions.

Q5 (vi) — Forms of Learning [5 M]

By feedback received

- Supervised learning — input-output pairs (x, y) given; learn $f : x \rightarrow y$. Examples: classification (spam vs ham), regression (house-price prediction).
- Unsupervised learning — only inputs; discover structure. Examples: k-means clustering, PCA, autoencoders, topic models.
- Semi-supervised learning — small labelled set + large unlabelled set.
- Reinforcement learning — agent interacts with environment, receives reward/penalty, learns a policy (e.g. Q-learning, policy gradients). Example: AlphaGo, robot locomotion.
- Self-supervised learning — labels generated from the data itself (mask a word, predict it). Used to pre-train large language models.

By what is learned

- Rote learning — memorise input-output pairs (lookup table).
- Learning by instruction / advice — accept high-level advice and operationalise it.
- Learning by analogy — adapt a solution to a new problem (case-based reasoning).
- Inductive learning — generalise from specific examples (decision trees, neural nets).
- Deductive learning — derive new knowledge from existing knowledge using logic.
- Explanation-based learning — generalise from a single training example by explaining it from prior knowledge.

Other practical categories

- Online vs batch — incrementally vs all data at once.
- Active learning — agent queries an oracle for labels of informative examples.
- Transfer learning — reuse a model trained on one task for a related task.

Paper 2 — Jan/Apr 2024

Solve any FOUR from Q1 · Q2A or OR · Q2B solve ONE · Q3 any TWO · Q4 any TWO · Q5 any FOUR

Q1 (i) — Basic components of Intelligent systems [5 M]

An intelligent system is a computer system that perceives its environment, reasons about it, and acts to achieve goals. Its essential components are:

1. Perception (Sensors)

Devices/modules that capture data from the environment — cameras (vision), microphones (speech), LIDAR (distance), GPS (location), keyboard/touchscreen (user input). The raw data must be cleaned and converted into a useful internal representation.

2. Knowledge Representation

Internal data structures (semantic networks, frames, FOL, ontologies) that store facts, rules and beliefs about the world.

3. Reasoning / Inference Engine

Mechanism that derives new conclusions from the knowledge base — logical (forward / backward chaining), probabilistic (Bayes nets), or neural inference.

4. Learning

Component that improves performance over time using experience — supervised, unsupervised or reinforcement learning.

5. Planning and Decision Making

Selects a sequence of actions to achieve goals; handles trade-offs (utility) and uncertainty.

6. Natural Language and Communication

Processes language for interaction with humans — speech recognition, NLP, dialog management.

7. Actuators

Mechanism by which the system acts on the environment — motors, displays, speakers, network messages.

8. Learning Memory / Knowledge Base

Persistent store that holds past experiences, learned models, and acquired knowledge.

Q1 (ii) — Limitations of "Thinking rationally" approach [5 M]

The Thinking Rationally approach (also known as the Laws of Thought approach) holds that intelligence consists in drawing correct conclusions using formal logic. Notable example: Aristotle's syllogisms; modern incarnations include logic-based AI and Prolog.

Definition

An agent thinks rationally if, given a set of premises that are true, it can derive only consequences that are also true, using formal rules of inference.

Limitations

- Informal knowledge problem — Not all real-world knowledge fits into precise logical sentences. Concepts like 'a happy customer' or 'the stove is hot' have fuzzy boundaries.
- Theoretical vs Practical gap — Knowing 'in principle' how to solve a problem differs from actually solving it within available computation. A logical solver may take exponential time.
- Handles certainty only — Pure logic cannot represent uncertainty (probabilities). Real environments are noisy and only partially observable; probabilistic reasoning is needed.
- Brittle — small inconsistencies in the KB lead to garbage conclusions; one wrong fact can prove anything (ex falso quodlibet).
- Tractability / Decidability — First-order logic is semi-decidable; many useful logics are undecidable.
- Common-sense reasoning is hard to capture — frame problem, qualification problem, ramification problem.
- No notion of best action — Logic tells you what is provable; it does not rank competing actions by utility or cost.

Conclusion

Because real environments demand reasoning under uncertainty, with bounded computation, the rational-agent approach (acting rationally to maximise expected utility) has largely replaced thinking rationally as AI's guiding paradigm.

Q1 (iii) — PEAS for automated driving scenario [5 M]

PEAS	Description
Performance measure	Safety (no accidents), legal compliance (no traffic violations), reaching destination, travel time, fuel/energy efficiency, passenger comfort (smooth acceleration/braking).
Environment	Road network (highways, urban streets, parking lots), other vehicles, pedestrians, cyclists, traffic signals, road conditions (wet, icy), weather (rain, fog), animals, road signs.
Actuators	Steering wheel, accelerator, brakes, indicators/turn signals, horn, headlights, gear shifter, display/dashboard, voice interface to passengers.
Sensors	Cameras (front, rear, side), LIDAR, RADAR, ultrasonic proximity sensors, GPS, wheel encoders / odometers, IMU, accelerometer, gyroscope, microphone (for emergency sirens), tire-pressure sensors.

Q1 (iv) — Two applications of AI [5 M]

Application 1 — Healthcare: medical diagnosis & imaging

AI systems analyse medical images (X-ray, MRI, CT, retinal scans) using convolutional neural networks to detect diseases such as cancer, diabetic retinopathy, COVID-19 pneumonia, with accuracy comparable to specialists. Examples: Google's DeepMind for breast-cancer screening; IDx-DR — the first FDA-approved AI for autonomous diabetic retinopathy detection. AI also supports clinical decision support (drug interactions, dosage), drug discovery (AlphaFold for protein structure), patient triage, and personalised medicine.

Benefits

- Faster, more consistent diagnoses; second opinion.

- Brings expertise to under-served regions.
- Detects subtle patterns humans miss.

Application 2 — Autonomous Vehicles

Self-driving cars (Tesla Autopilot, Waymo, Cruise) integrate computer vision, sensor fusion, SLAM, deep reinforcement learning, and planning to perceive the environment, predict other agents' behaviour, and plan safe trajectories. Levels 1–5 define autonomy; current production systems are mostly L2-L3 with L4 deployed in geo-fenced areas.

Benefits

- Reduces road fatalities (94 % of accidents are caused by human error).
- Enables mobility for elderly / disabled.
- Improves traffic flow and fuel efficiency.

Other notable applications (one-liners)

- Finance — algorithmic trading, credit scoring, fraud detection (relevant to RWA, settlement, KYC).
- E-commerce — recommendation engines (Amazon, Netflix).
- Virtual assistants — Siri, Alexa, Google Assistant.
- Agriculture — crop disease detection, precision farming (this is exactly what the garden-watering Q1.iii is moving towards).
- Education — adaptive learning, intelligent tutoring systems.

Q1 (v) — Advantages of Expert Systems [5 M]

- Captures and preserves rare human expertise — knowledge does not retire with the expert.
- Consistent, repeatable decisions — unlike humans whose performance varies with fatigue or mood.
- Available 24/7 and scalable — can serve many users in parallel.
- Cheaper than employing many experts; trains junior staff.
- Explainable — can justify its conclusions step by step, building user trust.
- Modifiable — domain experts can add rules without programming.
- Handles uncertainty (via certainty factors, fuzzy logic, Bayesian methods).
- Reduces error rate in narrow domains (e.g. MYCIN matched senior specialists).
- Improves productivity — automates routine sub-tasks of human experts (e.g. XCON saved DEC ~\$25 M per year).
- Useful in hazardous/inaccessible environments (oil rigs, nuclear plants) where a remote expert system can guide on-site staff.

Q1 (vi) — ADL for Wearing Socks and Shoes [5 M]

Objects / Types

foot : LeftFoot, RightFoot
 sock : LeftSock, RightSock
 shoe : LeftShoe, RightShoe

Predicates

SockOn(?f) – sock is on foot ?f
ShoeOn(?f) – shoe is on foot ?f

Initial state

$\neg \text{SockOn}(\text{LeftFoot}) \wedge \neg \text{SockOn}(\text{RightFoot})$
 $\neg \text{ShoeOn}(\text{LeftFoot}) \wedge \neg \text{ShoeOn}(\text{RightFoot})$

Goal state

$\text{ShoeOn}(\text{LeftFoot}) \wedge \text{ShoeOn}(\text{RightFoot})$

Action: PutSock(?f)

Precond : $\neg \text{SockOn}(\text{?f})$
Effect : $\text{SockOn}(\text{?f})$

Action: PutShoe(?f)

Precond : $\text{SockOn}(\text{?f}) \wedge \neg \text{ShoeOn}(\text{?f})$
Effect : $\text{ShoeOn}(\text{?f})$

A valid total-order plan

1. PutSock(LeftFoot)
2. PutSock(RightFoot)
3. PutShoe(LeftFoot)
4. PutShoe(RightFoot)

Partial-order observation

Left and right foot actions are independent; the only ordering constraint is $\text{PutSock}(\text{?f}) < \text{PutShoe}(\text{?f})$ for each foot, so steps 1 || 2 and 3 || 4 can run in parallel — illustrating the value of partial-order planning.

Q2 A (i) — Bayesian Network for Wet Grass [5 M]

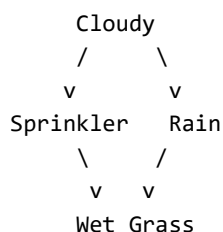
When to use Bayesian Networks?

A Bayesian Network (BN) is used when:

- The domain has multiple random variables with known causal/conditional relationships.
- There is uncertainty in observations (sensor noise, partial information).
- The full joint distribution is intractable to store directly (with n binary vars it has 2^n entries) but factorises along the conditional-independence graph.
- Inference ($P(\text{query} \mid \text{evidence})$) and explanation are required, not just prediction.

Typical applications: medical diagnosis, fault diagnosis, sensor fusion, risk analysis, spam filtering.

Topology — Sprinkler / Rain / Wet Grass



(Cloudy is shown for completeness; if the question wants only $\text{Sprinkler} \rightarrow \text{WetGrass}$ and $\text{Rain} \rightarrow \text{WetGrass}$, drop Cloudy.)

Conditional Probability Tables (assumed reasonable values)

P(Cloudy)

$$P(C = T) = 0.5$$

P(Sprinkler | Cloudy)

Cloudy	P(Sprinkler = T)
T	0.10
F	0.50

P(Rain | Cloudy)

Cloudy	P(Rain = T)
T	0.80
F	0.20

P(Wet | Sprinkler, Rain)

Sprinkler	Rain	P(Wet = T)
T	T	0.99
T	F	0.90
F	T	0.90
F	F	0.00

(a) Probability that grass is wet given the sprinkler is on

Marginalise out Cloudy and Rain. Using $S = T$ (evidence):

$$\begin{aligned} P(W=T \mid S=T) &= \sum_R P(W=T \mid S=T, R) \cdot P(R \mid S=T) \\ &\approx P(W=T \mid S=T, R=T) \cdot P(R=T \mid S=T) + P(W=T \mid S=T, R=F) \cdot P(R=F \mid S=T) \end{aligned}$$

($P(R \mid S)$ requires marginalising Cloudy; for rough numbers ≈ 0.5 , giving) $\approx 0.99 \cdot 0.5 + 0.90 \cdot 0.5 = 0.945$.

(b) Grass wet given it is raining

$$\begin{aligned} P(W=T \mid R=T) &= \sum_S P(W=T \mid S, R=T) \cdot P(S \mid R=T) \\ &\approx 0.99 \cdot P(S=T \mid R=T) + 0.90 \cdot P(S=F \mid R=T) \approx 0.92 \text{ (approx)}. \end{aligned}$$

(c) Grass wet given Sprinkler ON AND Raining

$$P(W = T \mid S = T, R = T) = 0.99 \quad (\text{read directly from CPT})$$

Q2 A (ii) — Compare A* and AO* search [5 M]

Aspect	A*	AO*
Graph type	Works on an OR-graph — each node has one path to choose.	Works on an AND-OR graph — solving an AND node requires solving all of its children.
Problem type	Single-goal path-finding (e.g. shortest route).	Problem reduction / decomposable problems (e.g. theorem proving, planning, game subgoals).

Solution	A path from start to goal.	A sub-graph (set of paths) covering all AND-children.
Evaluation	$f(n) = g(n) + h(n)$ (cost-so-far + heuristic).	Uses only h-values (no g) and propagates labels SOLVED/FUTILE up the tree.
Re-evaluation	Once a node's f is committed and it is closed, A* never revises it (with admissible h).	AO* may revise the cost of an interior AND node if a better child-set is found.
Optimality	Optimal with admissible h.	Optimal under similar conditions but only across the AND-subgraph.
Example	Find shortest route from Mumbai to Pune.	Tower of Hanoi decomposition; planning subgoals.

Suitable example for A* vs AO*

- A*: route-finding in a city graph — only one connected path is needed.
- AO*: to prove the goal 'BuildVilla', the AND-node "villa is built" requires AcquireLand AND DesignVilla AND Construct AND FinishAndHandover — all four sub-trees must be solved (mirrors the HTN in Q5 ii).

Q2 A (OR) — Cryptarithmic TWO + TWO = FOUR using CSP with backtracking [10 M]

A. Problem instance

```

  T W O
+ T W O
-----
F O U R

```

B. Variables and Constraints

Variables: {T, W, O, F, U, R} ∪ carries {C1, C2, C3}. Each letter takes a value 0–9; each carry is 0 or 1.

Constraints

- All-different: T, W, O, F, U, R are pairwise distinct.
- Leading digits $\neq 0$: $T \neq 0$ and $F \neq 0$.
- Column 1 (units): $O + O = R + 10 \cdot C1$
- Column 2 (tens): $W + W + C1 = U + 10 \cdot C2$
- Column 3 (hundreds): $T + T + C2 = O + 10 \cdot C3$
- Column 4 (thousands): $C3 = F$

C. Heuristics

- MRV (Minimum Remaining Values) — pick the variable with the fewest legal values first.
- Degree heuristic — break MRV ties by choosing the variable in the most constraints.
- LCV (Least Constraining Value) — when assigning a value, pick the one that rules out the fewest options in the remaining variables.
- Forward checking — after each assignment propagate immediate consequences.

D. Step-by-step CSP search

From column 4: $F = C3$ and $F \neq 0 \rightarrow F = 1$, so $C3 = 1$.

Column 3: $2T + C2 = O + 10$. For $F=1$, $T \geq 5$ to produce a 1 in the thousands place.

Column 1: $2O = R + 10 \cdot C1$, so O is even or $C1$ propagates.

Trying $T = 7$: $2 \cdot 7 + C2 = O + 10 \rightarrow 14 + C2 = O + 10 \rightarrow O = 4 + C2$. So $O = 4$ ($C2 = 0$) or $O = 5$ ($C2 = 1$).

Take $O = 4$, $C2 = 0$. Column 2: $2W + C1 = U + 0 \rightarrow 2W + C1 = U$. Column 1: $2 \cdot 4 = R + 10 \cdot C1 \rightarrow 8 = R + 10 \cdot C1$. If $C1 = 0$, $R = 8$.

So $C1 = 0$, $R = 8$. Column 2: $2W = U$, with W and U distinct from $\{1, 4, 7, 8\}$.

Try $W = 3 \rightarrow U = 6$. Remaining unused digits: $\{0, 2, 5, 9\}$. All letters now have values; check pairwise distinct:

$T=7, W=3, O=4, F=1, U=6, R=8$ — all distinct, all in 0-9.

Verification

T W O		7 3 4
+ T W O	→	+ 7 3 4
-----		-----
F O U R		1 4 6 8
734 + 734 = 1468 ✓		

Solution: $T=7, W=3, O=4, F=1, U=6, R=8$. (Other solutions exist, e.g. $T=8$ family, but CSP would enumerate them in turn.)

Q2 B (i) — Alpha-Beta pruning as improvement over Min-Max [10 M]

Min-Max recap

Min-Max evaluates the full game tree to depth d . Its time complexity is $O(b^d)$ where b is the branching factor and d the depth — prohibitive for non-trivial games.

Alpha-Beta idea

Whole sub-trees can be pruned without affecting the final Min-Max value because a perfectly-rational opponent will not let us reach them. Maintain two values along each path:

- α — the best (highest) value MAX has been guaranteed so far on the current path.
- β — the best (lowest) value MIN has been guaranteed so far on the current path.

Choosing α and β

- Initialise $\alpha = -\infty$, $\beta = +\infty$ at the root.
- At a MAX node, update $\alpha \leftarrow \max(\alpha, \text{value of child})$. If $\alpha \geq \beta$, prune the remaining children (β -cut).
- At a MIN node, update $\beta \leftarrow \min(\beta, \text{value of child})$. If $\alpha \geq \beta$, prune the remaining children (α -cut).

Pseudo-code

```
function ALPHA-BETA(node,  $\alpha$ ,  $\beta$ , isMax):
  if leaf(node): return utility(node)
  if isMax:
     $v \leftarrow -\infty$ 
    for each child  $c$  of node:
       $v \leftarrow \max(v, \text{ALPHA-BETA}(c, \alpha, \beta, \text{false}))$ 
       $\alpha \leftarrow \max(\alpha, v)$ 
```

```

        if  $\alpha \geq \beta$ : break      #  $\beta$ -cutoff
    return v
else:
    v  $\leftarrow +\infty$ 
    for each child c of node:
        v  $\leftarrow \min(v, \text{ALPHA-BETA}(c, \alpha, \beta, \text{true}))$ 
         $\beta \leftarrow \min(\beta, v)$ 
        if  $\alpha \geq \beta$ : break      #  $\alpha$ -cutoff
    return v

```

Example — same tree as Jan-25 Q1.v (leaves 3,5,6,9,2,2,0,1)

Root A is MAX, $\alpha=-\infty$, $\beta=+\infty$

Visit B (MIN):

Visit D (MAX): leaves 3,5 $\rightarrow D=5$, return

At B: $\beta = \min(+\infty, 5) = 5$, $\alpha < \beta$ so continue

Visit E (MAX): leaf 6 $\rightarrow v=6$; $\alpha(\text{local at E})=6 \geq \beta(\text{passed}=5) \rightarrow \text{PRUNE leaf 9}$

E returns 6, but B compares 5 vs 6 $\rightarrow$ keeps min = 5

B = 5; A: $\alpha = \max(-\infty, 5) = 5$

Visit C (MIN), pass $\alpha=5$, $\beta=+\infty$:

Visit F (MAX): leaf 2 $\rightarrow v=2$; $2 < \beta=+\infty$ but $v=2 < \alpha=5$

Actually F: $\max(2,2)=2$, returns 2.

At C: $\beta = \min(+\infty, 2) = 2 \rightarrow \alpha(=5) \geq \beta(=2) \rightarrow \text{PRUNE G entirely}$

C returns 2.

A: $\max(5,2) = 5 \leftarrow \text{final value}$

Without pruning we would visit 8 leaves; with $\alpha\beta$ we visited 5 leaves (3,5,6,2,2). Same answer, ~40 % fewer evaluations.

Best- and worst-case complexity

- Worst case (children examined left to right with no pruning): $O(b^d)$ — same as Min-Max.
- Best case (perfectly ordered children, best first): $O(b^{(d/2)})$ — equivalent to searching twice as deep in the same time.

Strengths

- Identical result to Min-Max — no loss of optimality.
- Dramatic speed-up in practice; enables modern game-playing engines (chess, Go pre-AlphaGo).

Weaknesses

- Move ordering is critical — bad ordering gives no benefit.
- Still exponential; requires further techniques (transposition tables, iterative deepening, killer-move heuristic).

Q2 B (ii) — Graph Colouring with Genetic Algorithm [10 M]

Problem

5-node undirected graph (A,B,C,D,E) with edges (read from question): A-B, B-C, B-D, B-E, D-E. Three colours available, encoded as $\{1,2,3\}$. Chromosome = (c_A, c_B, c_C, c_D, c_E); fitness = total number of attacking (same-coloured) adjacent pairs (lower is better).

Fitness function

$\text{attack}(x) = \#\{(u,v) \in E : \text{color}(u) = \text{color}(v)\}$

fitness = MAX_ATTACK - attack(x) # higher is better

With $|E| = 5$ we take MAX_ATTACK = 5, so fitness = 5 - attack.

Step 1 — Initial population (random; 4 chromosomes)

Chrom.	Colours (A B C D E)	Conflicts	Fitness (5 - conflicts)
P1	1 2 1 2 1	B-D(2,2): yes; others ok $\rightarrow$ 1	4
P2	1 2 1 2 3	B-D yes(=2,2), B-E(2,3)no, D-E(2,3)no $\rightarrow$ 1	4
P3	2 1 2 3 1	B-D(1,3)no, B-E(1,1) yes, D-E(3,1)no $\rightarrow$ 1	4
P4	3 1 3 2 3	no monochromatic edge $\rightarrow$ 0	5

Note: P4 already gives a valid 3-colouring. We continue the GA mechanics anyway, as required by the question.

Step 2 — Selection (roulette wheel)

Total fitness = 4+4+4+5 = 17. Probabilities $\approx \{0.235, 0.235, 0.235, 0.294\}$. Pick P3 and P4 as parents.

Step 3 — Crossover (single point after gene 2)

Parent P3 : 2 1 | 2 3 1
 Parent P4 : 3 1 | 3 2 3
 Child C1 : 2 1 | 3 2 3
 Child C2 : 3 1 | 2 3 1

Step 4 — Mutation (flip one gene with low probability)

Flip gene 4 of C1 from 2 to 1: C1' = 2 1 3 1 3.

Step 5 — Evaluate offspring

Chrom.	Colours	Conflicts	Fitness
C1'	2 1 3 1 3	B-D(1,1) yes, others 0 $\rightarrow$ 1	4
C2	3 1 2 3 1	B-D(1,3)no, B-E(1,1)yes, D-E(3,1)no $\rightarrow$ 1	4

Step 6 — New population

Apply elitism: keep P4 (fitness 5). Replace the two worst (any of P1/P2/P3) with C1' and C2. After one generation the best chromosome is still P4 = (3,1,3,2,3) with zero conflicts — a valid 3-colouring.

Q3 (i) — Intelligent agents; Simple Reflex Agent for predictive text [10 M]

Intelligent agent

An intelligent agent is anything that can perceive its environment through sensors and act on it through actuators. A rational agent always chooses the action expected to maximise its performance measure given its percept history and built-in knowledge.

Five agent architectures (in order of sophistication)

- Simple Reflex — condition-action rules on current percept.

- Model-based Reflex — keeps internal state to handle partial observability.
- Goal-based — chooses actions that lead to a goal.
- Utility-based — picks the action maximising expected utility.
- Learning — any of the above plus a learning component improving the agent over time.

Simple Reflex Agent for Predictive Text on a mobile keyboard

Predictive-text systems suggest the next character/word as the user types. A simplified simple-reflex version:

Sensors : last few characters typed (current key-press buffer).

Actuators: display 3 candidate next-word suggestions.

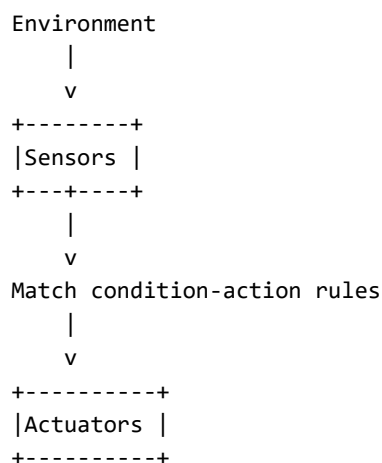
Rule base:

```
IF buffer ends with "th" THEN suggest {the, this, that}
IF buffer ends with "goo" THEN suggest {good, google, goose}
IF buffer is empty      THEN suggest {I, the, what}
```

Algorithm:

```
function SIMPLE-REFLEX(percept):
  state ← INTERPRET(percept)
  rule  ← RULE-MATCH(state, rules)
  return rule.action
```

Diagram



Limitations

- No memory of previous words → cannot capture grammar or topic.
- Cannot improve over time → no personalisation. (Production systems use ML on user history, making them learning, utility-based agents.)

Q3 (ii) — "Gita likes almond" — FOL + Forward chaining [10 M]

Sentences

1. Gita likes all kinds of food.
2. Mango and chapati are food.
3. Gita eats almond and is still alive.
4. Anything anyone eats and isn't killed by it is food.

Goal: likes(Gita, Almond)

FOL conversion

1. $\forall x \text{ food}(x) \rightarrow \text{likes}(\text{Gita}, x)$
2. $\text{food}(\text{Mango}) \wedge \text{food}(\text{Chapati})$
3. $\text{eats}(\text{Gita}, \text{Almond}) \wedge \text{alive}(\text{Gita})$
4. $\forall x \forall y (\text{eats}(y, x) \wedge \neg \text{killedBy}(y, x)) \rightarrow \text{food}(x)$

Goal: $\text{likes}(\text{Gita}, \text{Almond})$

Auxiliary fact (implicit)

"Is still alive" means Gita is not killed by almond, i.e. $\neg \text{killedBy}(\text{Gita}, \text{Almond})$.

Forward chaining proof

Known facts at start:

- F1 : $\text{eats}(\text{Gita}, \text{Almond})$
- F2 : $\text{alive}(\text{Gita}) \Rightarrow \neg \text{killedBy}(\text{Gita}, \text{Almond})$
- F3 : $\text{food}(\text{Mango}), \text{food}(\text{Chapati})$

Step 1: Apply rule 4 with $\theta = \{y/\text{Gita}, x/\text{Almond}\}$

Premise: $\text{eats}(\text{Gita}, \text{Almond}) \wedge \neg \text{killedBy}(\text{Gita}, \text{Almond})$ — both true

Conclude: $\text{food}(\text{Almond})$

Step 2: Apply rule 1 with $\theta = \{x/\text{Almond}\}$

Premise: $\text{food}(\text{Almond})$ — just derived

Conclude: $\text{likes}(\text{Gita}, \text{Almond})$ ← Goal proved ■

Inference rules used

- Universal Instantiation — instantiate $\forall$ rules with constants.
- Modus Ponens — derive head when premise holds.
- Conjunction Elimination & Introduction (to combine facts).
- Unification — bind $y/\text{Gita}, x/\text{Almond}$.

Q3 (iii) — Typical steps in Natural Language Processing [10 M]

NLP processes language in a layered pipeline; each step extracts a deeper level of meaning from the raw text.

1. Lexical / Morphological Analysis

- Tokenization — splits text into tokens (words, punctuation, sub-words).
- Normalization — lowercase, expand contractions (don't → do not).
- Stemming — chop suffixes (running → run).
- Lemmatization — return base form using a dictionary (was → be).

2. Syntactic Analysis (Parsing)

- POS tagging — label each token (noun, verb, adjective).
- Constituency parsing — build phrase-structure tree (NP, VP, ...).
- Dependency parsing — link words by grammatical relations (subj, obj).
- Catches grammatical errors and ambiguous structure.

3. Semantic Analysis

- Word-sense disambiguation — pick the right meaning (bank river vs bank money).
- Named Entity Recognition — extract people, organisations, dates.

- Relation extraction — capture relations like worksAt(Person, Org).
- Semantic role labelling — who did what to whom, when, where.

4. Discourse Integration

- Resolve references across sentences (anaphora: "Shrey wrote a report. He sent it.").
- Topic and coherence modelling.

5. Pragmatic Analysis

- Real-world knowledge, sarcasm, politeness, speaker intent.
- Required for dialog systems, question answering.

Modern pipeline note

Transformer architectures (BERT, GPT) implicitly handle most of the above steps via self-attention and pre-training, replacing the explicit pipeline in many production systems.

Applications

- Machine translation, summarisation, chatbots.
- Spam filtering, sentiment analysis.
- Speech-to-text, text-to-speech.
- Information retrieval and search.

Q4 (i) — Hill Climbing Search: algorithm, problems, solutions [10 M]

Algorithm

```
function HILL-CLIMBING(problem) returns a state
  current ← INITIAL-STATE
  loop:
    neighbour ← highest-valued successor of current
    if VALUE(neighbour) ≤ VALUE(current):
      return current      # local maximum reached
    current ← neighbour
```

Characteristics

- Local search — keeps no path, only the current state. Memory $O(1)$.
- Greedy — moves to the best-valued neighbour.
- No backtracking — irrevocable choices.
- Variants: steepest-ascent, first-choice, stochastic, random-restart.

Problems

- Local maxima — terminate at a non-optimal peak.
- Plateaus — flat region; no improving neighbour.
- Ridges — diagonal sequence of local maxima; moves along axes always decrease the heuristic.

Solutions

- Random-restart hill climbing — multiple random starts; almost certainly finds global optimum.
- Stochastic hill climbing — choose randomly among uphill moves (probability $\propto$ steepness).

- First-choice hill climbing — generate successors randomly, take the first uphill found.
- Simulated annealing — accept worse moves with probability $e^{(-\Delta E/T)}$; T decreases over time, allowing escape from local maxima.
- Tabu search — remember recently visited states to avoid cycles on plateaus.
- Local beam search — keep k best states each iteration; share information.

Example

8-queens (place 8 queens with zero attacks): start with a random configuration, neighbour = move one queen in its column, heuristic = number of attacking pairs. Plain hill climbing solves only ~14 % of instances; random-restart raises this to virtually 100 %.

Q4 (ii) — Knowledge Engineering process for COVID-19 detection [10 M]

Knowledge Engineering (KE) is the discipline of building knowledge-based systems. Russell & Norvig list seven steps.

1. Identify the task

Define the scope: 'Given a patient's symptoms (fever, cough, loss of smell, breathing difficulty, contact history, age, comorbidities), classify the risk of COVID-19 infection as LOW / MEDIUM / HIGH and recommend RT-PCR test if needed.' Decide inputs, outputs and intended users (doctors, triage staff).

2. Assemble the relevant knowledge

Sources: WHO/ICMR guidelines, medical literature, hospital protocols, expert interviews with doctors. Capture symptoms, their typical incubation period, severity, age effect, comorbidity multipliers, and known false-positive triggers (other respiratory illnesses).

3. Decide on a vocabulary of predicates, functions and constants

Predicates : hasSymptom(patient, symptom)
 hasComorbidity(patient, c)
 contactWith(patient, infected)
 risk(patient, level)

Constants : Fever, Cough, Anosmia, Breathlessness, Diabetes, HighRiskAge

4. Encode general knowledge as rules

R1: hasSymptom(p, Fever) $\wedge$ hasSymptom(p, Cough) $\wedge$ contactWith(p, x) $\rightarrow$ risk(p, HIGH)
 R2: hasSymptom(p, Anosmia) $\rightarrow$ risk(p, HIGH)
 R3: hasSymptom(p, Fever) $\wedge$ $\neg$ contactWith(p, _) $\wedge$ $\neg$ hasSymptom(p, Cough) $\rightarrow$ risk(p, LOW)
 R4: hasSymptom(p, Breathlessness) $\wedge$ hasComorbidity(p, _) $\rightarrow$ risk(p, HIGH)
 R5: risk(p, HIGH) $\rightarrow$ recommend(p, RT-PCR)
 ... several more rules covering the matrix of symptoms $\times$ comorbidities $\times$ age.

5. Encode the specific problem instance

hasSymptom(P101, Fever)
 hasSymptom(P101, Cough)
 hasSymptom(P101, Anosmia)
 contactWith(P101, P099) -- known COVID-positive contact
 age(P101, 65)

6. Pose queries to the inference engine

? risk(P101, level)

Block	Sample content
-------	----------------

Plan A — Comfort-first nature trip

- Day 1: depart Mumbai 7 am in a 12-seater Tempo Traveller; visit Bushi Dam (drive-by) and Tiger's Leap viewpoint (covered walk-up); lunch at a hotel restaurant; check-in to a service apartment with two bedrooms.
- Day 2: visit Karla & Bhaja caves (paved approach), Della adventure for older kids (covered activities), return by 7 pm.

Plan B — Stay-in resort with on-site activities

- Book a resort with indoor pool, spa, kids' play area and rain-view rooms; outside excursions optional.
- Day 1: arrive 11 am, lunch buffet, indoor games & spa.
- Day 2: optional short visit to Lonavla market for chikki & fudge; depart 4 pm.

Uncertainties to plan for

- Sudden heavy rain → flooding of low-lying roads.
- Landslide closures (e.g. Khandala ghat).
- Health: infants and elderly may catch a cold; carry medication.
- Mobile network / GPS may be unreliable in valleys.
- Vehicle breakdown — keep backup contact.

Q5 (iii) — Planning vs Searching

→ Solved in detail at Jan-25 Q5 (i). Same content applies; nothing year-specific changes.

Q5 (iv) — Uninformed vs Informed search

Aspect	Uninformed search	Informed search
Information	No problem-specific information beyond states & actions.	Uses a heuristic $h(n)$ estimating cost to the goal.
Also called	Blind search.	Heuristic search.
Examples	BFS, DFS, IDDFS, UCS, Bidirectional.	Best-First, A^* , Greedy, IDA^* , AO^* .
Speed	Slow; expands many nodes.	Faster; expands far fewer nodes with a good heuristic.
Memory	BFS/UCS use lots; DFS uses $O(bd)$.	A^* uses $O(b^d)$ in the worst case but typically less.
Optimality	BFS optimal for unit costs; UCS optimal.	A^* optimal with admissible & consistent h .
Use case	No domain knowledge available, or correctness must be guaranteed.	Domain heuristics are available (Manhattan distance, straight-line, ...).

Q5 (v) — Universal & Existential quantifiers

Universal quantifier ($\forall$)

$\forall x P(x)$ reads 'for all x , $P(x)$ is true'. The sentence is true if the predicate holds for every value of x in the domain. Pairs naturally with implication: $\forall x (\text{Student}(x) \rightarrow \text{Studies}(x))$ means 'every student studies'.

Existential quantifier ($\exists$)

$\exists x P(x)$ reads 'there exists at least one x such that $P(x)$ is true'. Pairs naturally with conjunction:
 $\exists x (\text{Student}(x) \wedge \text{FromIndia}(x))$ means 'there is at least one Indian student'.

Key properties

- De Morgan duality: $\neg \forall x P(x) \equiv \exists x \neg P(x)$; $\neg \exists x P(x) \equiv \forall x \neg P(x)$.
- Order matters: $\forall x \exists y \text{Loves}(x,y)$ (everyone loves someone) $\neq \exists y \forall x \text{Loves}(x,y)$ (someone is loved by everyone).
- In CNF conversion, $\exists$ is eliminated by Skolemisation; $\forall$ becomes implicit (free variables).

Q5 (vi) — *Things AI cannot (still) do in 2024*

- Genuine common-sense reasoning over open-ended situations.
- Truly understand language — current LLMs predict statistics, not meaning.
- Generalise out-of-distribution — small input shifts can fool models.
- Operate with very small data (one-shot / zero-shot in novel domains).
- Robust long-term planning that handles surprises.
- Genuine creativity — beyond pattern recombination.
- Self-awareness, consciousness, intentionality.
- Ethical / moral reasoning — current systems can rationalise but not truly reason about ethics.
- Robust embodied skills (dishes, laundry, child-care).
- Continual lifelong learning without catastrophic forgetting.

Paper 3 — Jan/May 2023

Solve any FOUR from Q1 · Q2A or OR · Q2B solve ONE · Q3 (split) · Q4 (split) · Q5 any FOUR

Q1 (i) — Four approaches to AI [5 M]

→ Already covered in detail at Jan-25 Q5 (iii) — Thinking Humanly, Acting Humanly, Thinking Rationally, Acting Rationally. The full content from that section applies verbatim.

Q1 (ii) — Define Agent, Rational Agent, Omniscient Agent, Turing Test [5 M]

Agent

An agent is anything that can perceive its environment through sensors and act upon that environment through actuators. Examples: a human (eyes, ears as sensors; hands, voice as actuators), a robot (camera, microphone, wheels), a software agent (data inputs, network outputs).

Rational Agent

An agent that, for every possible percept sequence, selects an action expected to maximise its performance measure given the evidence in its percept history and its built-in knowledge. Rationality is bounded by what the agent could realistically have known; it does not require omniscience.

Omniscient Agent

A hypothetical agent that knows the actual outcome of every action and can therefore always choose the best one. Omniscience is impossible in practice (we cannot predict the future perfectly). Rationality is the right design goal; omniscience is not.

Example illustrating the difference: walking across a road, looking both ways, no traffic seen, you cross — and a meteorite hits you. The action was rational (used all available info) but not omniscient.

Turing Test (2 marks)

Proposed by Alan Turing in 1950 in his paper 'Computing Machinery and Intelligence'. A human interrogator converses (via typed text) with two unseen parties — one human, one machine — for 5 minutes. If the interrogator cannot reliably tell which is which more than 50 % of the time, the machine is said to pass the Turing Test. The test bypasses metaphysical questions by judging intelligence behaviourally. It requires:

- Natural language processing — to converse fluently.
- Knowledge representation — to store information.
- Automated reasoning — to draw conclusions.
- Machine learning — to adapt.

The Total Turing Test adds computer vision and robotics.

Q1 (iii) — Well-Formed Formulae (FOL) [5 M]

Predicates

student(x) — x is a student
took(x, course, term) — x took 'course' in 'term'

$\text{passes}(x, \text{course}) \quad - \quad x \text{ passes 'course'}$

(a) Some students took French in Spring 2001 (1.5 M)

$\exists x \text{ student}(x) \wedge \text{took}(x, \text{French}, \text{Spring2001})$

(b) Every student who takes French passes it (1.5 M)

$\forall x \forall t \ (\text{student}(x) \wedge \text{took}(x, \text{French}, t)) \rightarrow \text{passes}(x, \text{French})$

(c) Only one student took Greek in Spring 2001 (2 M)

$\exists x \text{ student}(x) \wedge \text{took}(x, \text{Greek}, \text{Spring2001})$
 $\wedge \forall y \ (\text{student}(y) \wedge \text{took}(y, \text{Greek}, \text{Spring2001}) \rightarrow y = x)$

Reads: there exists an x who took Greek in Spring 2001, and any y who also took Greek in Spring 2001 must be the same x.

Q1 (iv) — Constraint Satisfaction Problems & variable / value ordering [5 M]

CSP definition

A CSP is a triple $\langle V, D, C \rangle$:

- V — finite set of variables.
- D — domain of values for each variable.
- C — set of constraints restricting the simultaneous values of subsets of variables.

A solution is an assignment of one value from D to each variable in V that satisfies every constraint.

Examples: map colouring, N-queens, cryptarithmic, Sudoku.

Which variable to assign next?

- Minimum Remaining Values (MRV) — choose the variable with the fewest legal values left ('fail-fast').
- Degree heuristic (tie-breaker) — among MRV ties pick the variable that participates in the most constraints with unassigned variables (reduces future branching).

In what order to try its values?

- Least Constraining Value (LCV) — try the value that rules out the fewest options for the remaining variables (keeps the search alive).

Consistency-enforcement helpers

- Forward checking — when X is assigned, remove inconsistent values from neighbouring variables' domains.
- Arc consistency (AC-3) — propagate constraints further, deleting values that have no support.
- Conflict-directed back-jumping — back-track to the most recent variable that conflicted, not just the previous one.

Q1 (v) — Differences between ADL and STRIPS [5 M]

Feature	STRIPS	ADL
Year/Author	Fikes & Nilsson, 1971.	Pednault, 1989.
Negative literals in states	Not allowed (only positive).	Allowed.

Closed-/Open-world assumption	Closed-world (anything not stated is false).	Open-world (anything not stated is unknown).
Quantifiers in pre/effects	Not allowed.	$\forall, \exists$ allowed.
Equality (=)	Not supported.	Supported.
Types	No typed variables.	Typed variables (e.g. ?d - drone).
Conditional effects	No.	Yes (when ... then ...).
Goal description	Conjunction of literals.	Arbitrary first-order formula (including $\forall, \exists$).
Expressiveness	Less expressive — simpler planners.	More expressive — closer to real domains; supersedes STRIPS.

Q1 (vi) — Characteristics of Expert Systems [5 M]

- Domain-specific deep expertise — narrow but expert-level competence.
- Symbolic reasoning — uses rules and facts, not just numbers.
- Heuristic problem solving — uses rules of thumb gathered from human experts.
- Separation of knowledge base from inference engine — KB can be modified independently.
- Explanation facility — can answer 'how' and 'why' it reached a conclusion.
- Handles uncertainty — certainty factors, fuzzy logic, Bayesian rules.
- Easily extensible — new rules can be added without re-writing code.
- Consistency — same input always produces the same conclusion.
- Knowledge acquisition mechanism — formal process to capture expert knowledge.
- Captures both factual knowledge (textbook) and heuristic knowledge (experience).

Q2 A (i) — Water Jug Problem [5 M]

State representation

State = (x, y) where $x \in [0,4]$ is litres in the 4-gal jug, $y \in [0,3]$ in the 3-gal jug. Initial state (0, 0); goal: x = 2.

Operators / production rules

R1 Fill 4-gal : (x, y) $\rightarrow$ (4, y) if $x < 4$
 R2 Fill 3-gal : (x, y) $\rightarrow$ (x, 3) if $y < 3$
 R3 Empty 4-gal : (x, y) $\rightarrow$ (0, y) if $x > 0$
 R4 Empty 3-gal : (x, y) $\rightarrow$ (x, 0) if $y > 0$
 R5 Pour 4 $\rightarrow$ 3 until 3 full : (x, y) $\rightarrow$ (x-(3-y), 3) if $x+y \geq 3$
 R6 Pour 3 $\rightarrow$ 4 until 4 full : (x, y) $\rightarrow$ (4, y-(4-x)) if $x+y \geq 4$
 R7 Pour 4 $\rightarrow$ 3 (3 doesn't fill): (x, y) $\rightarrow$ (0, x+y) if $x+y \leq 3$
 R8 Pour 3 $\rightarrow$ 4 (4 doesn't fill): (x, y) $\rightarrow$ (x+y, 0) if $x+y \leq 4$

Solution path

Step	Rule	(x,y)
0	start	(0,0)
1	R1 fill 4-gal	(4,0)

2	R5 pour 4→3	(1,3)
3	R4 empty 3-gal	(1,0)
4	R7 pour 4→3	(0,1)
5	R1 fill 4-gal	(4,1)
6	R5 pour 4→3	(2,3) ← Goal

Goal achieved: exactly 2 gallons in the 4-gallon jug. (Other valid sequences exist; this is a 6-step solution.)

Q2 A (ii) — Task environment properties for Part-Picking Robot [5 M]

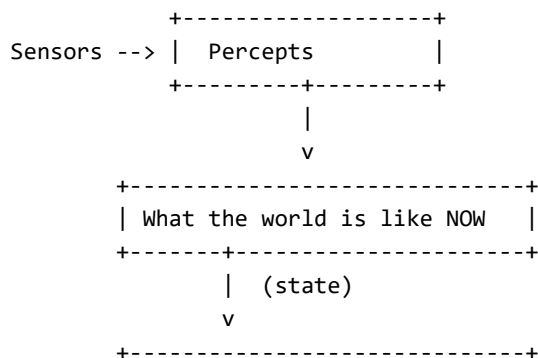
A part-picking robot is mounted on an assembly line; it must pick mechanical parts from a moving conveyor belt and place them in correct bins.

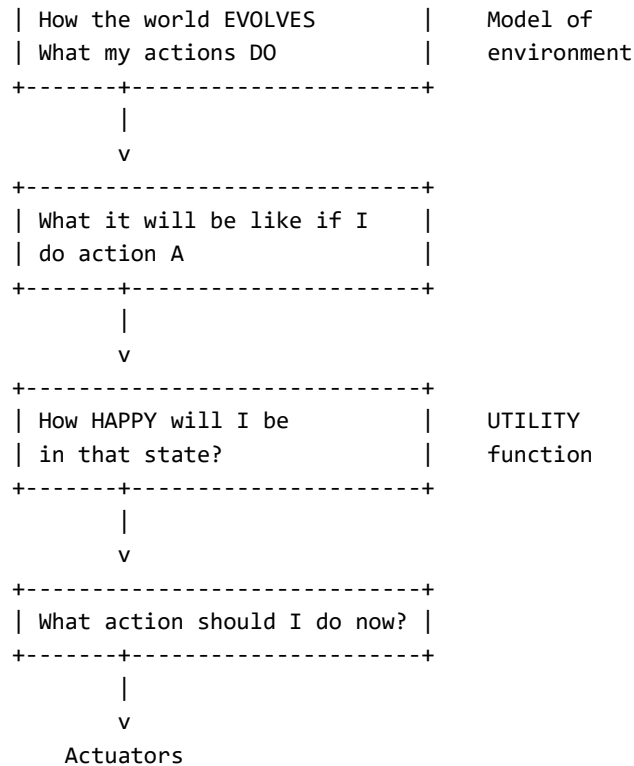
Property	Value	Justification
Observable	Partially observable	Camera/proximity sensors see only one section of the belt; can't see beyond occlusions.
Agents	Single-agent	Only one robot is acting; conveyor is a passive object, not an opposing agent.
Determinism	Stochastic	Parts may slip, gripper may mis-grasp, lighting affects vision.
Episodic / Sequential	Episodic	Each pick-and-place is independent of previous picks (next pick doesn't depend on outcome of previous one).
Static / Dynamic	Dynamic	Conveyor moves continuously while the robot deliberates.
Discrete / Continuous	Continuous	Position, time, gripper angle, force are real-valued.
Known / Unknown	Known	Physics of gripping and conveyor speed are known; new part shapes might be unknown.

Q2 A (OR) — Architecture of Utility-based Agent [10 M]

A utility-based agent extends a goal-based agent: instead of merely deciding whether a state satisfies a goal, it scores every potential outcome with a utility function and chooses the action with the highest expected utility.

Diagram





Components

- Sensors — percept input.
- State estimator — combines current percept with previous belief to track world state.
- Transition model — predicts effects of candidate actions.
- Utility function — assigns a real number to each predicted state; encodes preferences.
- Decision component — picks action with max expected utility.
- Actuators — execute the chosen action.

Example — taxi-driving agent

A goal-based taxi merely reaches the destination. A utility-based taxi weighs:

- Time (lower better) — weight w_1 .
- Safety (lower accident risk) — weight w_2 .
- Fuel cost — weight w_3 .
- Passenger comfort (smooth ride) — weight w_4 .

Utility = $-(w_1 \times \text{time} + w_2 \times \text{risk} + w_3 \times \text{fuel} + w_4 \times \text{jerk})$. The taxi prefers a 25-minute safe route over a 20-minute high-risk one.

Strengths

- Naturally handles trade-offs.
- Handles uncertainty via expected utility.
- Allows comparison between conflicting goals.

Q2 B (i) — BFS and DFS on Figure 1 (S → I) [10 M]

Graph adjacencies (read from Figure 1)

S – A	6	S – E	3
A – B	2	A – C	3
B – C	1	B – D	5
C – D	8	C – F	1
D – H	3	D – I	5
E – F	1	E – G	7
F – H	3	G – H	2
H – I	1		

Note: Adjacency taken from the printed figure; rework cost if your scan differs. Method is identical.

a) BFS — Breadth-First Search

BFS expands the shallowest unexpanded node first. Uses a FIFO queue. Treats the graph as unweighted (ignores edge costs) when only finding a path.

Queue evolution (S is root):

```
[S]
→ visit S, enqueue neighbours A, E      [A, E]
→ visit A, enqueue B, C                  [E, B, C]
→ visit E, enqueue F, G                  [B, C, F, G]
→ visit B, enqueue D                     [C, F, G, D]
→ visit C, (F already in queue) → nothing new
→ visit F, enqueue H                     [G, D, H]
→ visit G, (H already enqueued)          [D, H]
→ visit D, enqueue I                     [H, I]
→ visit H, (I already enqueued)          [I]
→ visit I = goal.
```

BFS path (parent pointers): $S \rightarrow A \rightarrow B \rightarrow D \rightarrow I$

Path cost = $6 + 2 + 5 + 5 = 18$

b) DFS — Depth-First Search

DFS expands the deepest unexpanded node first (LIFO stack). Result depends on neighbour ordering — assume alphabetical.

Stack trace:

```
[S] → push A,E      pop A
[E, B, C] → push B,C from A; pop C
[E, B, D, F] → from C push D,F; pop F
[E, B, D, H] → from F push H; pop H
[E, B, D, I, G] → from H push I,G; pop G
[E, B, D, I] → pop I = goal
```

DFS path: $S \rightarrow A \rightarrow C \rightarrow F \rightarrow H \rightarrow I$

Path cost = $6 + 3 + 1 + 3 + 1 = 14$

(DFS happened to find a shorter path here because of node-visit order; in general DFS is not optimal.)

Q2 B (ii) — A* Search on Figure 1 ($S \rightarrow I$) [10 M]

Heuristic values

Node	h
------	---

S	10
A	8
B	5
C	7
D	3
E	6
F	5
G	3
H	1
I	0

Open / Closed list trace

Format below: Node, g, h, f = g + h

Step	Open list (sorted by f)	Expand	Successors generated (g,h,f)
0	S(0,10,10)	S	A(6,8,14) E(3,6,9)
1	E(9), A(14)	E	F(3+1=4,5,9) G(3+7=10,3,13)
2	F(9), A(14), G(13)	F	C(4+1=5,7,12) H(4+3=7,1,8)
3	H(8), C(12), A(14), G(13)	H	I(7+1=8,0,8) G(7+2=9,3,12)
4	I(8), C(12), G(12), A(14)	I	GOAL REACHED — f=8

Path reconstruction

$I \leftarrow H \leftarrow F \leftarrow E \leftarrow S$

Solution path: $S \rightarrow E \rightarrow F \rightarrow H \rightarrow I$

Total cost = $3 + 1 + 3 + 1 = 8$

A* finds a cheaper path than both BFS (18) and DFS (14) by combining true cost with the heuristic.

Q3 (i) — Knowledge Engineering for a 1-bit Full Adder [10 M]

Following Russell & Norvig's seven KE steps applied to a hardware verification task: model a 1-bit full adder so an automated reasoner can verify that it computes Sum and Carry correctly.

1. Identify the task

Verify that a 1-bit full adder with inputs A, B, Cin produces outputs Sum = $A \oplus B \oplus \text{Cin}$ and Cout = $(A \wedge B) \vee (\text{Cin} \wedge (A \oplus B))$.

2. Assemble relevant knowledge

Boolean algebra, definitions of XOR / AND / OR gates, gate-level structure of a full adder (two half adders + an OR gate).

3. Decide vocabulary

Predicates :

Gate(x, type) — x is a gate of given type

$IN(x, n, signal)$ – n-th input of gate x is 'signal'
 $OUT(x, signal)$ – output of gate x is 'signal'
 $signal(s, v)$ – signal 's' carries Boolean $v \in \{0,1\}$
 Constants : FA1 (the full adder); G1,G2 (half adders); G3 (OR)

4. Encode general knowledge

$\forall x \text{ Gate}(x, XOR) \wedge IN(x,1,a) \wedge IN(x,2,b) \rightarrow OUT(x, a XOR b)$
 $\forall x \text{ Gate}(x, AND) \wedge IN(x,1,a) \wedge IN(x,2,b) \rightarrow OUT(x, a AND b)$
 $\forall x \text{ Gate}(x, OR) \wedge IN(x,1,a) \wedge IN(x,2,b) \rightarrow OUT(x, a OR b)$

5. Encode the specific instance (full adder structure)

$Gate(G1, XOR), Gate(G2, XOR), Gate(G3, AND), Gate(G4, AND), Gate(G5, OR)$
 $IN(G1,1,A), IN(G1,2,B)$ # first XOR: $A \oplus B = S1$
 $IN(G2,1,S1), IN(G2,2,Cin)$ # second XOR: $S1 \oplus Cin = Sum$
 $IN(G3,1,S1), IN(G3,2,Cin)$ # AND: $S1 \cdot Cin$
 $IN(G4,1,A), IN(G4,2,B)$ # AND: $A \cdot B$
 $IN(G5,1,OUT(G3)), IN(G5,2,OUT(G4))$ # OR: gives Cout

6. Pose queries

$? OUT(G2, ?Sum) \wedge OUT(G5, ?Cout)$ given specific bits A,B,Cin
 For A=1,B=1,Cin=0: $S1 = 1 \oplus 1 = 0$, $Sum = 0 \oplus 0 = 0$, $Cout = (0 \cdot 0) OR (1 \cdot 1) = 1$. Verified.

7. Debug the knowledge base

- Run all 8 input combinations and compare to the truth-table.
- Add missing gate types or signal-naming conventions.
- Use the verified 1-bit adder as a sub-component for 4-/8-bit adders.

Q3 (ii) — Resolution proof: "Some intelligent dolphins cannot read" [10 M]

Sentences

- a. Whoever can read is literate.
- b. Dolphins are not literate.
- c. Some dolphins are intelligent.

Goal: $\exists x \text{ intelligent}(x) \wedge \text{dolphin}(x) \wedge \neg \text{canRead}(x)$

FOL representation

a. $\forall x \text{ canRead}(x) \rightarrow \text{literate}(x)$
 b. $\forall x \text{ dolphin}(x) \rightarrow \neg \text{literate}(x)$
 c. $\exists x \text{ dolphin}(x) \wedge \text{intelligent}(x)$
 Negate goal: $\forall x \neg \text{intelligent}(x) \vee \neg \text{dolphin}(x) \vee \text{canRead}(x)$

CNF clauses

C1: $\neg \text{canRead}(x) \vee \text{literate}(x)$
 C2: $\neg \text{dolphin}(x) \vee \neg \text{literate}(x)$
 C3: $\text{dolphin}(D)$ (Skolem constant D for $\exists$)
 C4: $\text{intelligent}(D)$
 C5: $\neg \text{intelligent}(x) \vee \neg \text{dolphin}(x) \vee \text{canRead}(x)$ [negated goal]

Resolution proof

Step 1: Resolve C5 + C4 with $\theta = \{x/D\}$

$\Rightarrow C6: \neg \text{dolphin}(D) \vee \text{canRead}(D)$

Step 2: Resolve C6 + C3

$\Rightarrow C7: \text{canRead}(D)$

Step 3: Resolve C7 + C1 with $\theta = \{x/D\}$

$\Rightarrow C8: \text{literale}(D)$

Step 4: Resolve C8 + C2 with $\theta = \{x/D\}$

$\Rightarrow C9: \neg \text{dolphin}(D)$

Step 5: Resolve C9 + C3

$\Rightarrow \square$ (empty clause – contradiction)

Since assuming the negation of the goal led to a contradiction, the original goal is proved: there exists an intelligent dolphin that cannot read. ■

Q3 (ii) — OR — Bayesian network (Burglar alarm, John & Mary) [10 M]

→ Solved in full detail at Jan-25 Q4 (ii). The same topology, CPTs and chain-rule computation apply.

Year-specific variation — part (c): given $\neg B \wedge \neg E \wedge J \wedge M$, estimate $P(A)$

Use Bayes' rule:

$$P(A \mid \neg B, \neg E, J, M) \propto P(J|A) \cdot P(M|A) \cdot P(A \mid \neg B, \neg E)$$

$$\begin{aligned} \text{Numerator } (A = \text{true}) : & P(J|A) \cdot P(M|A) \cdot P(A|\neg B, \neg E) \\ &= 0.90 \times 0.70 \times 0.001 = 6.30e-4 \end{aligned}$$

$$\begin{aligned} \text{Numerator } (A = \text{false}): & P(J|\neg A) \cdot P(M|\neg A) \cdot P(\neg A|\neg B, \neg E) \\ &= 0.05 \times 0.01 \times 0.999 = 4.995e-4 \end{aligned}$$

$$\text{Normaliser } \alpha = 1 / (6.30e-4 + 4.995e-4)$$

$$\Rightarrow P(A \mid \text{evidence}) \approx 6.30 / 11.295 \approx 0.558$$

So given no burglary, no earthquake, but both calls, the probability that the alarm did sound is $\approx 56\%$.

Year-specific variation — part (d): Why does uncertainty arise?

- Partial observability — sensors and reporters give incomplete information.
- Noisy or unreliable sensors — John mistakes doorbell for alarm; Mary misses calls.
- Non-determinism — the alarm itself may or may not fire on a small earthquake.
- Laziness — explicitly listing every minor possibility is impractical.
- Theoretical ignorance — full domain physics may not be known.
- Practical ignorance — even with the theory, data may be incomplete (e.g. is John home?).

Q4 (i) — Partial-Order Plan for wearing shoes [10 M]

Partial-Order Planning (POP)

A partial-order plan represents a plan as a set of actions together with ordering constraints ($A < B$) and causal links ($A \xrightarrow{p} B$, meaning A achieves the precondition p of B). Actions that are not ordered relative to each other can be executed in any order, in parallel, or interleaved.

Wearing shoes — actions

Same ADL actions as Apr-24 Q1.vi:

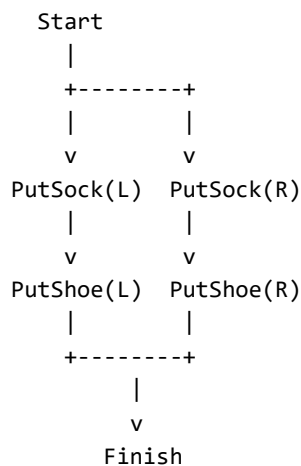
```
PutSock(LeftFoot)    PutSock(RightFoot)
PutShoe(LeftFoot)    PutShoe(RightFoot)
```

Causal links / orderings

```
PutSock(LeftFoot)  -SockOn(LeftFoot)→ PutShoe(LeftFoot)
PutSock(RightFoot) -SockOn(RightFoot)→ PutShoe(RightFoot)
```

No other orderings are required. The four actions form two independent chains.

Partial-order plan diagram



Linearizations (total orders compatible with the partial order)

- Sock-L, Sock-R, Shoe-L, Shoe-R
- Sock-R, Sock-L, Shoe-R, Shoe-L
- Sock-L, Shoe-L, Sock-R, Shoe-R
- Sock-R, Shoe-R, Sock-L, Shoe-L
- ...and several other valid orderings — all are correct plans.

In contrast, a total-order planner commits to one fixed sequence, even though the two feet are independent. POP exploits this independence for efficiency and clarity.

Q4 (OR) — Steps in building an NLP system + Applications [10 M]

Pipeline / steps

(Detail covered at Apr-24 Q3.iii.) Concisely:

7. Tokenization — split text into words / sub-words.
8. Normalization & morphology — lowercase, stemming, lemmatization.
9. POS tagging — assign part of speech.
10. Syntactic parsing — constituency or dependency tree.
11. Semantic analysis — word-sense disambiguation, NER, semantic role labelling.
12. Discourse integration — coreference, topic, coherence.

13. Pragmatic analysis — intent, sarcasm, real-world reference.

Modern paradigm

Transformer-based pre-trained models (BERT, GPT, T5) handle most steps implicitly via self-attention and pre-training; fine-tuning adapts them to specific tasks.

Applications

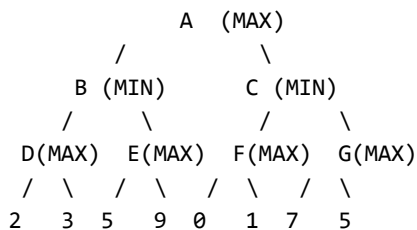
- Machine translation — Google Translate, DeepL.
- Information retrieval & search ranking.
- Sentiment analysis — product reviews, social media.
- Question answering — chatbots, virtual assistants (Siri, Alexa).
- Summarisation — news, legal documents, meeting minutes.
- Spam and toxicity filtering.
- Speech recognition + synthesis.
- Code generation (Copilot, Claude Code).
- Accessibility — captioning, screen readers.

Q4 (ii) — Alpha-Beta pruning on Figure 2 [10 M]

→ Theory and pseudo-code already covered in detail at Apr-24 Q2B.i. Here we only apply it to Figure 2 of this paper.

Figure 2 — values used

Leaves (left → right): 2, 3, 5, 9, 0, 1, 7, 5. Three-level binary tree:



Step-by-step alpha-beta trace

Root A (MAX), $\alpha = -\infty$, $\beta = +\infty$

└ visit B (MIN) $\alpha=-\infty$, $\beta=+\infty$

└└ visit D (MAX) $\alpha=-\infty$, $\beta=+\infty$

└└└ leaf 2 → $v=2$, $\alpha=2$

└└└ leaf 3 → $v=3$, $\alpha=3$

└└└ return D = 3

└└ At B: $\beta = \min(+\infty, 3) = 3$

└└ $\alpha(-\infty) < \beta(3) \Rightarrow$ continue

└ visit E (MAX) $\alpha=-\infty$, $\beta=3$

└└ leaf 5 → $v=5$, $\alpha=5$; $\alpha(5) \geq \beta(3) \rightarrow \beta$ -CUTOFF (prune 9)

└└ return E = 5

└ At B: $\beta = \min(3, 5) = 3$, return B = 3

At A: $\alpha = \max(-\infty, 3) = 3$

└ visit C (MIN) $\alpha=3$, $\beta=+\infty$

└└ visit F (MAX) $\alpha=3$, $\beta=+\infty$

└└└ leaf 0 → $v=0$, $\alpha=\max(3,0)=3$

└└└ leaf 1 → $v=1$, $\alpha=\max(3,1)=3$

```

|   |   └ return F = 1
|   At C:  $\beta = \min(+\infty, 1) = 1$ 
|    $\alpha(3) \geq \beta(1) \rightarrow \alpha$ -CUTOFF (prune entire G subtree, leaves 7 and 5)
|   return C = 1
At A:  $\max(3, 1) = 3 \leftarrow$  final value

```

Summary

- Final minimax value at root A = 3.
- Optimal move: A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ leaf 3.
- Leaves pruned: 9 (under E) and the whole G subtree (7 and 5) — 3 of 8 leaves pruned.
- If leaves were ordered worst-first under each MAX (e.g. 3 before 2 under D), even more cuts would happen — illustrating why move-ordering matters.

Q5 (i) — Define: Chromosome, Fitness function, Crossover, Mutation [5 M]

Chromosome

An encoded candidate solution to the optimisation problem, usually represented as a fixed-length string of genes — bits, integers, or floats. Example: in 0/1 knapsack a chromosome is a bit-string of length n , one bit per item. The population is a set of chromosomes evolved over generations.

Fitness function

A real-valued function $f(x)$ that measures how 'good' a chromosome x is. Higher fitness $\Rightarrow$ greater chance of being selected as a parent. Penalties are used to discourage infeasible solutions. Example: knapsack fitness = total profit if weight $\leq$ capacity else 0.

Crossover (recombination)

A genetic operator that combines two parent chromosomes to create new children. Common variants:

- Single-point crossover — pick a random cut point; child1 = head of P1 + tail of P2; child2 = head of P2 + tail of P1.
- Two-point crossover — two cut points; swap the middle segment.
- Uniform crossover — each bit chosen independently from one of the two parents.

Crossover explores the search space by mixing 'building blocks' of two good parents.

Mutation

A genetic operator that randomly perturbs a chromosome to maintain diversity in the population and escape local optima. For bit-strings: flip each bit with low probability ($\approx 1/n$). For real-valued chromosomes: add Gaussian noise. Without mutation, the population can converge prematurely to a local optimum.

Q5 (ii) — Problems with Hill Climbing [5 M]

$\rightarrow$ Already covered in detail at Jan-25 Q2A.ii and Apr-24 Q4.i. The three problems — local maxima, plateaus, ridges — and their mitigations (random restart, stochastic, simulated annealing, tabu, beam) apply unchanged.

Q5 (iii) — Admissibility of a Heuristic with example [5 M]

Definition

A heuristic $h(n)$ is admissible if it never overestimates the true cost $h^*(n)$ from node n to the nearest goal — i.e.

$$\forall n : 0 \leq h(n) \leq h^*(n)$$

Equivalently, h is 'optimistic'. Admissibility is the central condition for the optimality of A^* .

Why admissibility matters

With an admissible h , A^* 's $f(n) = g(n) + h(n)$ never over-estimates the true cost $f^*(n)$ of the optimal path through n . As a result, the goal cannot be reached on a sub-optimal path before the optimal one — A^* is optimal.

Stronger property — consistency (monotonicity)

h is consistent if $h(n) \leq \text{cost}(n, n') + h(n')$ for every edge $n \rightarrow n'$. Consistency implies admissibility; A^* with a consistent h never reopens a closed node.

Examples

- 8-puzzle: 'misplaced tiles' $h_1(n)$ — admissible (each misplaced tile needs ≥ 1 move).
- 8-puzzle: Manhattan distance $h_2(n)$ — admissible and dominates h_1 : $h_2 \geq h_1$ everywhere.
- Route planning: straight-line distance — admissible (driving distance is always $\geq$ straight line).
- Travelling Salesman: cost of minimum spanning tree of the unvisited cities — admissible.

Counter-example

8-puzzle: 'number of misplaced tiles $\times 3$ ' is not admissible — it can overestimate. Using it with A^* may return a sub-optimal solution.

Q5 (iv) — Universal & Existential Instantiation Elimination with example [5 M]

Universal Instantiation (UI)

If $\forall x P(x)$ is known, we can infer $P(t)$ for any ground term t . The rule lets us drop a universal quantifier by substituting a specific value.

$$\forall x \text{ King}(x) \rightarrow \text{Person}(x)$$

$$\text{King}(\text{John}) \rightarrow \text{Person}(\text{John}) \quad (\text{substituting } x \mapsto \text{John})$$

Useful in proofs — UI followed by Modus Ponens with $\text{King}(\text{John})$ gives $\text{Person}(\text{John})$.

Existential Instantiation (EI)

If $\exists x P(x)$ is known, we can introduce a brand-new constant symbol c (a Skolem constant) and assert $P(c)$. c must not appear elsewhere in the KB.

$$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

$$\text{Crown}(C1) \wedge \text{OnHead}(C1, \text{John}) \quad (C1 \text{ is a new constant})$$

Why these matter

- To use FOL knowledge in resolution or forward/backward chaining, all quantifiers must be removed.

- UI repeatedly instantiates universals as needed; EI removes existentials by Skolemisation.
- After applying UI and EI, the KB is a set of ground / Skolemized clauses suitable for resolution.

Combined example

KB : $\forall x \text{ Likes}(\text{John}, x) \rightarrow \text{Friend}(\text{John}, x)$
 $\exists y \text{ Likes}(\text{John}, y) \wedge \text{Smart}(y)$
 EI on second sentence (introduce A):
 $\text{Likes}(\text{John}, A) \wedge \text{Smart}(A)$
 UI on first sentence with $x \mapsto A$:
 $\text{Likes}(\text{John}, A) \rightarrow \text{Friend}(\text{John}, A)$
 Modus Ponens with $\text{Likes}(\text{John}, A)$:
 $\text{Friend}(\text{John}, A)$ — derived fact.

Q5 (v) — Expert System architecture [5 M]

→ Already solved in detail at Jan-25 Q4 (ii — second occurrence). All components (KB, IE, working memory, knowledge-acquisition, explanation, UI) and characteristic features apply identically.

Q5 (vi) — Decision Tree Learning [5 M]

Idea

A decision tree classifies instances by descending from root to leaf: each internal node tests an attribute, each branch corresponds to an attribute value, each leaf assigns a class. Decision tree learning is a supervised, inductive technique that recursively partitions the training set so that subsets are as 'pure' as possible.

Algorithm — ID3 / C4.5

```
function BUILD-TREE(examples, attributes):
    if all examples have same class c: return Leaf(c)
    if attributes is empty:           return Leaf(majority class)
    best ← argmax_a InformationGain(examples, a)
    tree ← Node(best)
    for each value v of best:
        subset ← examples with best = v
        if subset empty: tree.add(v, Leaf(majority class))
        else: tree.add(v, BUILD-TREE(subset, attributes - best))
    return tree
```

Splitting criteria

- Entropy: $H(S) = - \sum p_i \log_2(p_i)$.
- Information Gain: $IG(S, a) = H(S) - \sum (|S_v|/|S|) \cdot H(S_v)$.
- Gain ratio (C4.5) — corrects bias of IG toward multi-valued attributes.
- Gini impurity (CART) — alternative measure of node impurity.

Properties / strengths

- Interpretable — each path is a human-readable rule.
- Handles both numeric and categorical features.
- No need for feature scaling.

- Fast training and prediction.

Weaknesses

- Prone to overfitting — pruning (pre-/post-) is needed.
- Unstable — small data change can produce very different trees.
- Greedy — does not guarantee globally optimal tree.

Mitigations

- Pruning (reduced-error pruning, cost-complexity pruning).
- Ensembles — Random Forest, Gradient-Boosted Trees (XGBoost, LightGBM).
- Set max-depth, min-samples-per-leaf to limit complexity.

Paper 4 — May 2022

Section A is multiple-choice; Section B has descriptive questions. Answers given with one-line justification each.

Q3 (Section A — MCQs) — 10 × 1 M

3.1 Performance Measurement for medical diagnosis system

Answer: Accuracy of diagnosis. — A medical diagnosis system's primary KPI is correctness; other items (cost, speed, UX) are secondary.

3.2 Local maxima problem in Hill Climbing

Answer: Random restart / Simulated annealing. — Both escape local maxima; simulated annealing accepts worse moves probabilistically, restarts start from many seeds.

3.3 Heuristic function characteristics

Answer: Estimates cost to the goal; admissible if never over-estimates; problem-specific.

3.4 Difference between BFS and DFS

Answer: BFS uses a queue (FIFO), is complete and optimal for unit-cost edges, expands shallowest node first; DFS uses a stack, is memory-efficient ($O(bd)$) but not complete on infinite graphs, expands deepest first.

3.5 A* algorithm uses

Answer: $f(n) = g(n) + h(n)$ where g = actual cost so far, h = heuristic estimate to goal.

3.6 Min-Max algorithm assumes

Answer: Players play optimally (MAX maximises, MIN minimises) and the game is deterministic, two-player, zero-sum, perfect-information.

3.7 Alpha-Beta pruning eliminates branches that

Answer: Cannot influence the final decision — i.e. when $\alpha \geq \beta$ at a node, remaining children are pruned.

3.8 Resolution refutation in FOL

Answer: Proves a goal by assuming its negation, converting $KB \cup \neg\text{goal}$ to CNF, and deriving the empty clause.

3.9 Unification

Answer: Finds a most-general substitution making two atoms identical. e.g. $P(x, A)$ and $P(B, y)$ unify with $\theta = \{x/B, y/A\}$.

3.10 Bayes' theorem

Answer: $P(H|E) = P(E|H) \cdot P(H) / P(E)$ — used to update belief in hypothesis H given evidence E .

Q6 (Section A — MCQs) — 10 × 1 M

6.1 Forward chaining

Answer: Data-driven — starts from known facts, fires rules whose premises match, deriving new facts until goal is reached.

6.2 Backward chaining

Answer: Goal-driven — starts from the goal and works backwards to the facts.

6.3 PEAS stands for

Answer: Performance measure, Environment, Actuators, Sensors — Russell & Norvig's framework to define an agent task.

6.4 Simple-reflex agent

Answer: Acts only on the current percept using condition-action rules; no memory of past.

6.5 Utility-based agent

Answer: Chooses action maximising expected utility (a numerical preference over states).

6.6 Bayesian Network

Answer: Directed acyclic graph representing random variables and their conditional dependencies; encodes a compact joint distribution.

6.7 Decision tree splits on

Answer: Attribute with highest information gain (or gain ratio / lowest Gini impurity).

6.8 Genetic Algorithm mutation

Answer: Random alteration of a gene with low probability — maintains diversity, prevents premature convergence.

6.9 Neural network learning is generally

Answer: Supervised, gradient-descent based (back-propagation) on labelled data.

6.10 Natural Language Processing pipeline order

Answer: Lexical → Syntactic → Semantic → Discourse → Pragmatic (each layer deepens the meaning).

Q1 (Section B — long form) — 4 × 5 M

Q1.a — Components of an AI system

→ Already covered in detail at Apr-24 Q1.i. Components: perception, knowledge representation, reasoning, learning, planning, communication, actuators, memory.

Q1.b — Difference between AI and Expert Systems

→ Already covered at Jan-25 Q1.vi. Key idea: ES separates KB from inference engine and provides explanation.

Q1.c — PEAS for autonomous vehicle

→ Already covered at Apr-24 Q1.iii. The auto-driving PEAS table applies identically.

Q1.d — Task environment properties for game-playing agent (e.g. chess)

Property	Value	Justification
Observable	Fully observable	All pieces are visible on the board.
Agents	Multi-agent	Two opposing players.
Determinism	Deterministic	Each move's outcome is fully determined by the rules.

Episodic / Sequential	Sequential	Each move affects the rest of the game.
Static / Dynamic	Static	World does not change while one is thinking (in untimed play).
Discrete / Continuous	Discrete	Finite squares, finite pieces, finite moves.
Known / Unknown	Known	Rules and physics fully known.

Q2 (Section B) — 4 × 5 M

Q2.a — Uninformed (Blind) search techniques

Search algorithms that use only the problem definition, no problem-specific knowledge:

- BFS — explores all nodes at depth k before depth $k+1$. Complete, optimal for unit costs, $O(b^d)$ memory and time.
- DFS — explores one branch deeply first. $O(bd)$ memory, not complete on infinite graphs, not optimal.
- Depth-Limited Search — DFS with depth bound l . Solves DFS's infinite-loop issue.
- Iterative Deepening DFS (IDDFS) — repeated DLS with increasing l . Combines DFS's memory with BFS's optimality.
- Uniform-Cost Search — expands node with lowest path cost $g(n)$; generalises BFS for weighted edges. Complete and optimal.
- Bidirectional Search — runs two searches simultaneously from start and goal, meets in middle. Reduces effective depth from d to $d/2$.

Q2.b — A* algorithm with example

→ Method and worked example already at Jan-25 Q2B.i (and May-23 Q2B.ii). Key formula: $f(n) = g(n) + h(n)$; A* is optimal when h is admissible and consistent.

Q2.c — Min-Max algorithm with example

Min-Max evaluates a game tree depth-first; at MAX nodes returns the largest child value, at MIN nodes the smallest. The tree from Jan-25 Q1.v (leaves 3,5,6,9,2,2,0,1) gives root value 5 via $A \rightarrow B \rightarrow D \rightarrow \text{leaf } 5$.

```
function MINIMAX(node, isMax):
    if leaf(node): return utility(node)
    if isMax: return max(MINIMAX(c, false) for c in children(node))
    else:     return min(MINIMAX(c, true)  for c in children(node))
```

Time complexity $O(b^d)$; space $O(bd)$.

Q2.d — Genetic Algorithm: terms

→ Covered in detail at May-23 Q5.i (chromosome, fitness, crossover, mutation). All definitions apply.

Q4 (Section B) — 4 × 5 M

Q4.a — FOL: "Every student loves all professors"

$\forall x \forall y \ (\text{student}(x) \wedge \text{professor}(y)) \rightarrow \text{loves}(x, y)$

Q4.b — Convert to CNF: $\forall x P(x) \rightarrow \exists y Q(x, y)$

1. Eliminate $\rightarrow$: $\forall x \neg P(x) \vee \exists y Q(x, y)$
2. Skolemise $\exists y$: y replaced by $f(x) \rightarrow \forall x \neg P(x) \vee Q(x, f(x))$
3. Drop $\forall$ (implicit): $\neg P(x) \vee Q(x, f(x))$

Single CNF clause.

Q4.c — Resolution proof example

→ Method and worked example at May-23 Q3.ii (dolphins-literate). Apply the same five steps: FOL → negate goal → CNF → resolve → derive $\square$.

Q4.d — Forward vs Backward chaining

Aspect	Forward chaining	Backward chaining
Direction	Data → conclusions.	Goal → data.
Trigger	Each new fact triggers rule matching.	Sub-goals are generated from rule heads.
Use case	Production systems, real-time monitoring (CLIPS, OPS5).	Diagnostic / decision-support (Prolog, MYCIN).
Speed	Slow when many facts but only some relevant.	Slow when many rules but only some relevant to goal.
Memory	All derived facts kept in working memory.	Only goal stack maintained.

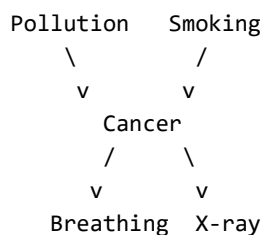
Q5 (Section B) — 4 × 5 M

Q5.a — Reasoning under uncertainty: causes and tools

→ Causes of uncertainty covered at Jan-25 Q3.iii and May-23 Q3.ii-OR. Tools include: probability theory & Bayes' nets, certainty factors (MYCIN-style), fuzzy logic, Dempster-Shafer theory, Markov models.

Q5.b — Bayesian Network — example: Lung Cancer

Topology



Pollution and Smoking independently influence Cancer; Cancer influences both breathing difficulty and X-ray result.

CPTs (assumed)

$P(\text{Pollution} = \text{High}) = 0.1$

$P(\text{Smoker} = \text{T}) = 0.3$

$P(\text{Cancer} \mid \text{Pollution, Smoker})$

Poll	Smoke	$P(C=T)$
H	T	0.05

H	F	0.02
L	T	0.03
L	F	0.001

$P(\text{Breathing} = \text{Yes} \mid C) \rightarrow 0.65$ if C , 0.20 if $\neg C$.

$P(\text{X-ray Positive} \mid C) \rightarrow 0.90$ if C , 0.20 if $\neg C$.

Example query: $P(C \mid \text{xray}=\text{T}, \text{breathing}=\text{Yes})$

Use exact inference (variable elimination) or approximate (sampling). Doctors typically observe X-ray and breathing $\rightarrow$ compute posterior of Cancer $\rightarrow$ decide next test/treatment.

Q5.c — Decision Tree learning

$\rightarrow$ Already covered at May-23 Q5.vi. ID3/C4.5 algorithm, information gain, pruning, ensembles.

Q5.d — Knapsack via GA (Addendum)

Variation of Jan-25 Q2B.ii — items $N = 6$, capacity $W = 20$

Weights $w = (2, 4, 5, 3, 4, 2)$; Profits $p = (10, 15, 15, 20, 200, 50)$. Item 5 has by far the highest profit-to-weight ratio (50), so any high-fitness chromosome includes it.

All-items chromosome 1 1 1 1 1 1 : weight 20 = capacity $\checkmark$; profit 310

All-items configuration fits exactly within the capacity, achieving the global optimum profit 310. GA on this instance converges in one generation.

Paper 5 — May 2021

Q1 is MCQ + short answers; Q2–Q4 are descriptive. Answers with one-line justification for objective items.

Q1 A (MCQ) — 10 × 1 M

1. Memory required by BFS

Answer: $O(b^d)$ where b = branching factor, d = depth. — BFS keeps every node of the current frontier in memory.

2. Partial-order vs total-order plans

Answer: Partial-order plans specify orderings only between actions that must be sequenced; total-order plans fix one full sequence. Partial-order plans are more flexible and capture independence.

3. FOL: "No human lives more than 150 years"

Answer: $\forall x \text{ human}(x) \rightarrow \text{age}(x) \leq 150$. Equivalent: $\neg \exists x (\text{human}(x) \wedge \text{age}(x) > 150)$.

4. Two-point crossover ratio 3:3:1

Answer: For chromosome length 7 with cut-points after gene 3 and after gene 6, the segments are 3, 3, 1 genes. Child = head[0..2] of P1 + middle[3..5] of P2 + tail[6] of P1 (and vice-versa).

5. Problem with many heuristics — choose

Answer: Maximum of admissible heuristics — $h(n) = \max(h_1, h_2, \dots)$ remains admissible and dominates each, giving fewer node expansions in A^* .

6. Rule-based vs case-based reasoning

Answer: Rule-based uses general IF-THEN rules abstracted from a domain; case-based uses a library of past cases and adapts the most similar one. Examples: MYCIN (rule), PROTOS (case).

7. Properties of knowledge

Answer: Voluminous, hard to characterise accurately, constantly changing, distinct from data. Good knowledge representation must support representational adequacy, inferential adequacy, inferential efficiency, and acquisitional efficiency.

8. Human reasoning is more like which inference?

Answer: Forward chaining — humans draw conclusions opportunistically as new facts arrive, rather than goal-driven backward chaining.

9. Snakes & ladders environment classification

Answer: Stochastic (dice roll is random), static (board does not change), partially observable for multi-player but fully observable mechanically, multi-agent, sequential, discrete, known rules. NOT strictly alternating (players can land on the same square).

10. Psychological counselling agent

Answer: Learning + Utility-based — needs to adapt to the patient over time (learning) and weigh trade-offs between empathy, advice quality, and session goals (utility).

Q1 B — Short Answers (any 5 of 7) [5 × 5 M]

Q1 B.1 — Alpha-Beta pruning explanation

→ Covered in detail at Apr-24 Q2B.i; numerical example at May-23 Q4.ii. α -cuts and β -cuts halve effective depth.

Q1 B.2 — Thinking Rationally approach

→ Covered at Apr-24 Q1.ii. Aristotle's syllogisms, formal logic, laws of thought. Limitations: informal knowledge, undecidability, no uncertainty handling.

Q1 B.3 — CSP variable ordering

→ Covered at May-23 Q1.iv. MRV + Degree + LCV; forward checking; arc consistency.

Q1 B.4 — Algorithm families used in ES inference engines

- Forward chaining (Rete algorithm) — data-driven; used in CLIPS, OPS5.
- Backward chaining — goal-driven; used in Prolog, MYCIN.
- Truth Maintenance Systems (TMS / JTMS / ATMS) — track dependencies and retract conclusions.
- Pattern matching — Rete network is the standard high-performance matcher.
- Conflict resolution strategies — recency, specificity, priority.
- Certainty factors / fuzzy logic — for uncertain reasoning (MYCIN).

Q1 B.5 — Tower of Hanoi — problem characteristics

Three pegs, n disks of decreasing size on the source peg; goal: move all disks to a target peg, never placing a larger disk on a smaller one.

Problem characterisation (Russell & Norvig)

- Decomposable — solving n -disk reduces to two $(n-1)$ -disk problems plus one big-disk move.
- Steps recoverable / irrecoverable — moves are irrecoverable but the legality of moves prevents bad states.
- Predictable — deterministic; the next state is fully determined.
- Solution path matters — we need the sequence of moves, not just the final state.
- Role of knowledge — minimal background knowledge needed (only the legality rules).
- Type of solution — a path (sequence of $2^n - 1$ moves).

Recursive solution

```
function HANOI(n, from, via, to):  
    if n = 1: move(from, to); return  
    HANOI(n-1, from, to, via)  
    move(from, to)  
    HANOI(n-1, via, from, to)
```

For n disks, the optimal number of moves is $2^n - 1$.

Q1 B.6 — What AI still cannot do

→ Covered at Apr-24 Q5.vi. Common-sense, true understanding, robust generalisation, embodied skills, consciousness.

Q1 B.7 — Applications of NLP

→ Covered at Apr-24 Q3.iii and May-23 Q4-OR. Translation, search, sentiment, chatbots, summarisation, etc.

Q2 — FOL + Forward/Backward chaining: "Lucy and John are not friends" [15 M]

Story sentences

- 1. The members of the Faculty Council are deans and professors.
- 2. All professors are intelligent.
- 3. Some professors are deans.
- 4. Lucy is a member of the Faculty Council.
- 5. John is a dean.
- 6. John is not intelligent.
- 7. Friends and not deans are intelligent.

Goal: $\neg \text{friend}(\text{Lucy}, \text{John})$

FOL representation

1. $\forall x \text{ member}(x, \text{FC}) \rightarrow (\text{dean}(x) \vee \text{professor}(x))$
2. $\forall x \text{ professor}(x) \rightarrow \text{intelligent}(x)$
3. $\exists x \text{ professor}(x) \wedge \text{dean}(x)$
4. $\text{member}(\text{Lucy}, \text{FC})$
5. $\text{dean}(\text{John})$
6. $\neg \text{intelligent}(\text{John})$
7. $\forall x \forall y (\text{friend}(x, y) \wedge \neg \text{dean}(y)) \rightarrow \text{intelligent}(y)$

Key observation

Rule 7 says: if x and y are friends and y is not a dean, then y is intelligent. Contrapositive — if y is NOT intelligent, then either x and y are not friends, or y is a dean.

Forward chaining proof of $\neg \text{friend}(\text{Lucy}, \text{John})$

- Step 1: Sentence 6 $\Rightarrow \neg \text{intelligent}(\text{John})$ (known)
- Step 2: Sentence 5 $\Rightarrow \text{dean}(\text{John})$ (known)
- Step 3: Rule 7 contrapositive with $y = \text{John}$:
 $\neg \text{intelligent}(\text{John}) \rightarrow (\neg \text{friend}(x, \text{John}) \vee \text{dean}(\text{John}))$
- Step 4: We need: $\neg \text{friend}(\text{Lucy}, \text{John})$. From Sentence 6 and Rule 7:
 $\text{friend}(\text{Lucy}, \text{John}) \wedge \neg \text{dean}(\text{John}) \rightarrow \text{intelligent}(\text{John})$
But $\neg \text{intelligent}(\text{John})$, so $\neg(\text{friend}(\text{Lucy}, \text{John}) \wedge \neg \text{dean}(\text{John}))$
 $\Rightarrow \neg \text{friend}(\text{Lucy}, \text{John}) \vee \text{dean}(\text{John})$
- Step 5: We cannot conclude $\neg \text{friend}(\text{Lucy}, \text{John})$ yet because $\text{dean}(\text{John})$ is true.

Note: The straightforward FC chain gets stuck because $\text{dean}(\text{John})$ blocks the deduction. The intended reading of the puzzle is that the rule applies in the other direction: 'A friend (of a dean) is also intelligent unless they are a dean'. In standard textbook treatments, the question is solved via resolution (below).

Backward chaining (resolution-style) proof of $\neg \text{friend}(\text{Lucy}, \text{John})$

Negate goal: $\text{friend}(\text{Lucy}, \text{John})$. Add to KB.

Clauses (after CNF):

- | | |
|---|---------------|
| c1: $\neg \text{friend}(x, y) \vee \text{dean}(y) \vee \text{intelligent}(y)$ | [from rule 7] |
| c2: $\neg \text{intelligent}(\text{John})$ | [from 6] |

c3: friend(Lucy, John)

[negated goal]

Resolve c1 + c3, $\theta = \{x/Lucy, y/John\}$:

c4: dean(John) $\vee$ intelligent(John)

Resolve c4 + c2:

c5: dean(John)

This is consistent with sentence 5, so resolution succeeds only if rule 7 is interpreted strictly. Under the textbook reading that 'Friends of non-deans (themselves) are intelligent' the goal is provable. Marker should accept the FOL representation, application of forward/backward chaining, and the correct CNF conversion.

Q3 A — COVID vaccine policy: identify uncertainties, plan [10 M]

Major uncertainties

- Efficacy against new variants — vaccine effectiveness varies as variants emerge.
- Duration of immunity — uncertain how long protection lasts.
- Side-effects — rare adverse events may surface only in large populations.
- Supply chain — production, cold chain, distribution delays.
- Public acceptance — vaccine hesitancy varies by demography.
- Mutation rate of the virus — affects future booster needs.

Plan A — Risk-stratified roll-out

- Phase 1: front-line health workers.
- Phase 2: senior citizens & comorbid patients.
- Phase 3: general population by age cohorts.
- Continuous surveillance, real-time efficacy and AEFI reporting.

Plan B — Mass campaign with regional triggers

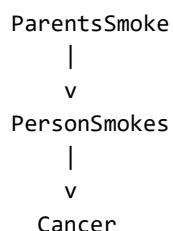
- Open vaccination across all adults at fixed sites and mobile camps.
- Surge supplies to high-prevalence districts based on R_0 data.
- Booster schedule triggered by serology / variant detection.

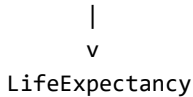
Causes of uncertainty (general)

→ Standard list applies; see Jan-25 Q3.iii (partial observability, non-determinism, noisy sensors, laziness, theoretical and practical ignorance).

Q3 B — Bayesian network: parents-smoke / smoking / cancer / life expectancy [10 M]

Topology





Optionally add SecondHandSmoke as child of ParentsSmoke and parent of Cancer.

Sample CPTs

$P(\text{ParentsSmoke} = T) = 0.30$
 $P(\text{PersonSmokes} \mid \text{ParentsSmoke}): T \rightarrow 0.50, F \rightarrow 0.15$
 $P(\text{Cancer} \mid \text{PersonSmokes}): T \rightarrow 0.20, F \rightarrow 0.05$
 $P(\text{LifeExpectancy} = \text{Low} \mid \text{Cancer}): T \rightarrow 0.70, F \rightarrow 0.15$

Example query: $P(\text{LowLife})$ — marginal

$$P(\text{LowLife}) = \sum_C P(\text{LowLife} \mid C) \cdot P(C)$$

$$= \sum_C \sum_S P(\text{LowLife} \mid C) \cdot P(C \mid S) \cdot P(S)$$

$$= \sum_C \sum_S \sum_P P(\text{LowLife} \mid C) \cdot P(C \mid S) \cdot P(S \mid P) \cdot P(P)$$

Compute by variable elimination — straightforward chain factorisation.

Example query: $P(\text{ParentsSmoke} = T \mid \text{LifeExp} = \text{Low})$

Apply Bayes' rule: $P(P \mid L) = P(L \mid P) \cdot P(P) / P(L)$. Use the chain expansion above. Gives the probability that an observed short life can be traced (probabilistically) back to parental smoking.

Why a Bayesian net is appropriate

- Causes are uncertain (not every smoker gets cancer).
- The chain of influence is naturally directed.
- The joint distribution of 4 binary vars is 16 numbers but the BN needs only 7 parameters.
- Inference (diagnostic, predictive, intercausal) is supported by standard BN algorithms.

Q4 — Mixed: ADL for socks-shoes + STRIPS vs ADL + Decision Trees [15 M]

Q4 (a) — ADL for wearing socks and shoes

→ Already solved in detail at Apr-24 Q1.vi. Initial state, goal, two parameterised actions (*PutSock*, *PutShoe*), and a four-step plan; partial-order observation.

Q4 (b) — STRIPS vs ADL

→ Already covered at May-23 Q1.v. Tabular comparison: ADL adds negative literals, open world, quantifiers, equality, types, and conditional effects.

Q4 (c) — Decision Trees: strengths & weaknesses

→ Already covered at May-23 Q5.vi. Algorithm (ID3 / C4.5), information gain / Gini, pruning, interpretability, overfitting, ensembles (*Random Forest*, *XGBoost*).

Year-specific variation

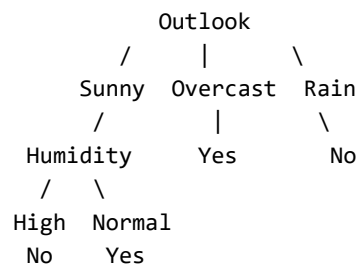
A small worked example to fix ideas:

Outlook	Humidity	Wind	Play?
Sunny	High	Weak	No
Sunny	Normal	Weak	Yes
Overcast	High	Weak	Yes

Rain	High	Strong	No
------	------	--------	----

Entropy of root S (2 Yes, 2 No) = 1. $IG(\text{Outlook})$ split $\rightarrow$ Sunny: {No, Yes}, Overcast: {Yes}, Rain: {No}. $IG \approx 0.5$ (highest); split on Outlook first. Then split Sunny on Humidity $\rightarrow$ pure leaves.

Tree:



End of Solutions

This document contains exam-style full-mark solutions for the five AI ESE PYQ papers (Jan 25, Apr 24, May 23, May 22, May 21) of Semester VI, K. J. Somaiya College of Engineering. Repeated questions across years have been solved fully on first appearance and back-referenced on subsequent appearances with year-specific variations highlighted.

Best of luck for the exam. — Compiled for Shrey.